

Launcher de Unificação de Jogos e Comunidade

**Kaio Damasceno de Oliveira¹, Mateus Hideki de Figueiredo Tamura²,
Matheus Barbosa Silva³, Rafael Pires Goncalves⁴, Renan Trajano da Conceição⁵,
Thiago Akira Damasceno Taniguchi⁶, Ariel Paulino Gonzaga⁷**

¹Instituto Federal de Educação, Ciência e Tecnologia
Câmpus São Paulo (IFSP) São Paulo – SP – Brazil

{kaio.d, h.tamura, matheus.barbosa1, goncalves.pires, renan.trajano,
akira.taniguchi, barbara.gonzaga}@aluno.ifsp.edu.br

1. Introdução

A indústria de jogos digitais tem apresentado crescimento contínuo nas últimas décadas, impulsionado principalmente pela expansão das plataformas de distribuição digital. Esse cenário possibilitou o surgimento de serviços como *Steam*, *Epic Games* e outras lojas virtuais, que permitem aos usuários adquirir e gerenciar uma grande quantidade de títulos. Como consequência, as bibliotecas digitais tornaram-se cada vez maiores e mais diversificadas, aumentando a complexidade na organização e no acesso aos jogos.

Dados recentes reforçam a dimensão desse mercado. De acordo com a pesquisa *Global Gamer Study*, realizada com 76 mil jogadores, aproximadamente 43% dos usuários utilizam a plataforma PC como principal meio para jogos digitais (Newzoo, 2024). Além disso, plataformas como a *Steam* já registraram mais de 42 milhões de usuários jogando simultaneamente em março de 2026 (SteamDB, 2026), evidenciando a escala e a relevância desses ambientes digitais.

Nesse contexto, surgem os chamados *launchers de jogos*, aplicações responsáveis por instalar, organizar e executar títulos digitais, oferecendo ao usuário uma interface centralizada para gerenciamento de sua biblioteca. No entanto, como cada plataforma possui seu próprio *launcher*, o usuário frequentemente precisa utilizar múltiplas aplicações distintas para acessar todos os seus jogos, o que resulta em um ambiente fragmentado e pouco eficiente.

Essa fragmentação gera diversos problemas, como a descentralização das bibliotecas, a divisão de progresso entre plataformas e a necessidade de alternar entre diferentes aplicações para acessar conteúdos. Além disso, o grande volume de jogos disponíveis dificulta a descoberta de títulos relevantes, tornando a experiência do usuário menos eficiente. Para lidar com esse problema, sistemas de recomendação têm sido amplamente utilizados em plataformas digitais, sendo capazes de sugerir conteúdos com base no comportamento e nas preferências dos usuários (MARINHO et al., 2015).

Entretanto, nas plataformas atuais, os dados utilizados para recomendação são limitados ao ecossistema de cada serviço, o que reduz a precisão das sugestões, uma vez que não consideram o comportamento completo do usuário em diferentes ambientes. Esse cenário evidencia a necessidade de soluções que integrem informações de múltiplas fontes, permitindo uma análise mais abrangente e eficiente.

1.1. Objetivos

1.1.1 Objetivo Geral

Desenvolver um *launcher* de jogos para *desktop* que permita organizar jogos locais, registrar automaticamente o tempo de execução e oferecer recomendações personalizadas baseadas no comportamento do usuário e na interação social entre jogadores.

1.1.2 Objetivos Específicos

- Implementar um sistema de biblioteca que permita adicionar e organizar jogos instalados localmente.
- Integrar a aplicação com a API da *Steam* para obtenção automática de informações sobre os jogos.

- Desenvolver um agente responsável por monitorar a execução dos jogos e registrar automaticamente o tempo de uso.
- Implementar um sistema de recomendação baseado em características dos jogos e no comportamento do usuário.
- Desenvolver funcionalidades sociais que permitam adicionar amigos e visualizar suas atividades.
- Criar um painel analítico para visualização de estatísticas de uso e preferências do jogador.

2. Referencial Teórico

2.1. Sistemas de Recomendação em *Python*

No contexto do *Salsilauncher*, um dos principais desafios é auxiliar o usuário na identificação de jogos relevantes dentro de sua própria biblioteca, reduzindo a necessidade de busca manual e melhorando a experiência de uso. Esse problema está diretamente relacionado ao grande volume de informações disponíveis e à ausência de mecanismos de curadoria personalizada, sendo amplamente tratado por sistemas de recomendação.

De acordo com Marinho et al. (MARINHO et al., 2015), sistemas de recomendação são responsáveis por sugerir itens relevantes com base em dados de interação dos usuários, utilizando informações como histórico de uso e preferências para prever interesses futuros. O estudo destaca que esses sistemas são aplicados em diversos contextos digitais, especialmente em ambientes com grande quantidade de conteúdo.

Entre as abordagens apresentadas, destaca-se a filtragem colaborativa, que utiliza dados de múltiplos usuários para identificar padrões de similaridade. Nesse modelo, assume-se que usuários com comportamentos semelhantes tendem a compartilhar interesses, permitindo a recomendação de novos itens com base nessas relações. Para isso, são utilizadas estruturas organizadas de dados e métricas matemáticas, como a similaridade entre vetores, que permitem quantificar o grau de proximidade entre elementos.

No *Salsilauncher*, essa abordagem é aplicada ao considerar dados como tempo de jogo e frequência de execução para representar a interação entre usuário e jogos. A partir dessas informações, o sistema pode identificar padrões de comportamento e calcular similaridades entre títulos, permitindo a geração de recomendações personalizadas. Dessa forma, o projeto se fundamenta diretamente nas técnicas apresentadas por Marinho et al., adaptando-as ao contexto específico de gerenciamento de bibliotecas de jogos digitais.

2.2. Desenvolvimento de Aplicações *Web* com *Frameworks Python*

O desenvolvimento do *Salsilauncher* envolve a integração de diferentes componentes responsáveis pela interface do usuário, processamento de dados e armazenamento de informações. Esse cenário exige uma arquitetura que permita comunicação eficiente entre as partes do sistema, garantindo desempenho e escalabilidade.

O estudo de análise comparativa de *frameworks web* (SCOTT; LEWIS, 2024) investiga a evolução do desenvolvimento de aplicações, destacando o papel de *frameworks* modernos na construção de sistemas mais eficientes e escaláveis. O trabalho evidencia que aplicações contemporâneas tendem a adotar arquiteturas baseadas em serviços, utilizando APIs para promover a separação entre *frontend* e *backend*.

Além disso, o artigo destaca o crescimento do uso de *frameworks Python*, ressaltando características como flexibilidade, produtividade e suporte a operações assíncronas. Essas propriedades tornam tais *frameworks* especialmente adequados para aplicações que necessitam lidar com múltiplas requisições e processamento contínuo de dados.

No *Salsilauncher*, esses conceitos são aplicados por meio da adoção de uma arquitetura baseada em API, na qual o *backend*, desenvolvido com *FastAPI*, é responsável por centralizar a lógica do sistema. Essa estrutura permite que o *frontend* interaja com o sistema de forma desacoplada, realizando operações como cadastro de jogos, recuperação de dados e solicitação de recomendações. Assim, o projeto segue diretamente as diretrizes apresentadas no estudo, garantindo organização e eficiência na comunicação entre seus componentes.

2.3. Algoritmos de Recomendação e Estruturação de Dados

A eficiência de sistemas de recomendação depende diretamente da forma como os dados são estruturados e dos algoritmos utilizados para processá-los. No *Salsilauncher*, esse desafio está presente na necessidade de representar as interações entre usuários e jogos de forma que permita identificar padrões e gerar recomendações relevantes.

O trabalho desenvolvido por Takahashi (TAKAHASHI, 2014) analisa diferentes algoritmos de recomendação e suas aplicações, destacando abordagens como filtragem colaborativa, filtragem baseada em conteúdo e sistemas híbridos. O estudo parte do cenário de crescimento do volume de dados digitais e da necessidade de extrair informações úteis a partir dessas interações.

A pesquisa demonstra que a escolha do algoritmo e a forma de organização dos dados são fatores determinantes para a qualidade das recomendações. Métodos baseados em conteúdo utilizam características dos itens e preferências do usuário, enquanto a filtragem colaborativa explora padrões de comportamento entre diferentes usuários. Além disso, o trabalho enfatiza a importância do tratamento adequado dos dados para garantir maior precisão nos resultados.

No nosso escopo, aplicados a ideia base na modelagem das interações entre usuários e jogos, utilizando dados como tempo de uso e frequência de execução. A partir dessa base, o sistema pode aplicar diferentes abordagens de recomendação para gerar sugestões personalizadas.

3. Materiais e Métodos

Esta seção apresenta os materiais e métodos utilizados no desenvolvimento do *Salsilauncher*. Inicialmente, são descritas as tecnologias e ferramentas empregadas na implementação do sistema. Em seguida, são detalhados os métodos adotados para a construção da aplicação, incluindo sua arquitetura, funcionamento e estratégias utilizadas para o desenvolvimento das funcionalidades propostas.

3.1. Materiais

3.1.1 Desenvolvimento *Front-end*

O *frontend* foi desenvolvido utilizando *React* e *TypeScript* (Meta Open Source, 2026; Microsoft, 2026b). A interface da aplicação é responsável pela visualização da biblioteca de jogos, interação com o usuário e envio de requisições ao *backend*.

A aplicação é empacotada como *software desktop* utilizando *Electron* (OpenJS Foundation, 2026), permitindo sua execução local e acesso direto a recursos do sistema operacional.

3.1.2 Desenvolvimento *Back-end*

O *backend* foi desenvolvido em *Python*, utilizando o *framework FastAPI* (Python Software Foundation, 2026; FastAPI, 2026). Ele é responsável pelo processamento das requisições, gerenciamento dos dados e implementação das funcionalidades do sistema.

Para persistência dos dados, foi utilizado o *PostgreSQL* (PostgreSQL Global Development Group, 2026), onde são armazenadas informações como jogos cadastrados, tempo de uso, avaliações e dados dos usuários.

3.1.3 APIs Externas

O sistema prevê integração com a *Steam Web API* (Valve Corporation, 2026), utilizada para obtenção de informações sobre jogos, como nome, descrição, categorias e dados associados à biblioteca do usuário. Essa integração permite que parte dos metadados seja preenchida automaticamente, reduzindo a necessidade de cadastro manual. A arquitetura do sistema também é elaborada de forma que possibilita a integração futura com outras plataformas que disponibilizem APIs públicas ou gratuitas.

Para a obtenção das informações dos jogos cadastrados na plataforma, o *Salsilauncher Server*-instância central da aplicação mantida em ambiente remoto - implementa um fluxo estruturado em *pipeline* de busca hierárquica. O processo de consulta e cadastro de metadados segue a sequência descrita abaixo:

1. **Consulta ao banco de dados local do servidor:** Toda vez que um usuário ou aplicação cliente solicita as informações de um jogo, o servidor realiza uma consulta inicial em seu próprio banco de dados relacional (*PostgreSQL*). Caso o registro seja encontrado, os dados são retornados imediatamente à aplicação cliente.
2. **Consulta às APIs externas em cadeia:** Caso o jogo não esteja registrado no banco de dados central, o sistema aciona sequencialmente as integrações com APIs externas de metadados. Atualmente, a busca é efetuada na *Steam Web API*.
3. **Persistência automática em banco (*caching*):** Ao obter os metadados de um jogo provenientes de uma API externa (como a *Steam*), o servidor salva automaticamente tais dados em seu banco local antes de responder à requisição. Além disso, quando um usuário realiza o cadastro manual de um jogo, as informações também são persistidas no servidor central.

Essa estratégia em *pipeline* reduz significativamente a quantidade de chamadas repetitivas às APIs externas, minimizando o risco de bloqueios por limite de requisições e otimizando o tempo de resposta para consultas futuras dos usuários.

3.1.4 Ferramentas de Desenvolvimento

Durante o desenvolvimento, foram utilizadas ferramentas como *Visual Studio Code* ([Microsoft, 2026c](#)), utilizado como ambiente de desenvolvimento, *GitHub* ([GitHub, 2026b](#)), empregado para versionamento de código, e *GitHub Projects* ([GitHub, 2026a](#)), utilizado para organização das tarefas. A comunicação e a organização da equipe foram realizadas por meio da plataforma *Discord* ([Discord, 2026](#)).

3.2. Métodos

3.2.1 Arquitetura do Sistema

O *Salsilauncher* foi concebido com base em uma arquitetura cliente-servidor descentralizada, na qual a aplicação cliente (*desktop*) interage com uma instância central da aplicação mantida em ambiente remoto, denominada *Salsilauncher Server*. Esta abordagem foi adotada para centralizar operações críticas que não devem ser executadas no lado do cliente por questões de segurança, desempenho e controle de acesso.

O *Salsilauncher Server* consiste em uma versão especializada do *backend* desenvolvida em *Python* com *FastAPI*, responsável por intermediar as comunicações com serviços externos e gerenciar as funcionalidades sociais do sistema. Entre as responsabilidades do servidor central, destacam-se:

- **Gerenciamento de requisições a APIs externas:** As requisições direcionadas à *Steam Web API* são centralizadas no servidor, que utiliza uma chave de API (*API key*) própria e protegida. Dessa forma, elimina-se a necessidade de o usuário fornecer ou autenticar sua própria chave de API para obter dados de jogos, garantindo maior segurança e praticidade.
- **Centralização do sistema social:** O servidor atua como repositório e distribuidor de informações de usuários. A aplicação cliente envia ao servidor dados como tempo de jogo acumulado, atividades em tempo real, informações de perfil e avaliações (*reviews*). O servidor armazena essas métricas e as disponibiliza aos demais usuários por meio do sistema de amizades, respeitando as configurações de privacidade.

3.2.2 Controle de Versão

O controle de versão do projeto é realizado com *Git* ([Git, 2026](#)), com o repositório hospedado no *GitHub* ([GitHub, 2026b](#)). Essa organização permite registrar o histórico de alterações do código, acompanhar as contribuições dos integrantes e facilitar a colaboração entre os membros da equipe.

Durante o desenvolvimento, os *commits* e *pull requests* seguem uma padronização interna, com o objetivo de manter o histórico do projeto organizado e facilitar a revisão das alterações realizadas.

3.2.3 Documentação

A documentação principal do projeto é produzida em $\text{\LaTeX}$, utilizando o *Overleaf* ([Overleaf, 2026](#)) como ambiente de edição colaborativa. As apresentações utilizadas nas entregas parciais são elaboradas com o *Google Slides* ([Google, 2026](#)). Essas ferramentas foram adotadas para organizar os artefatos acadêmicos do projeto e facilitar a edição colaborativa entre os integrantes da equipe.

3.2.4 Desenvolvimento Incremental

O desenvolvimento do projeto é realizado de forma incremental. Inicialmente, o foco está na construção da biblioteca de jogos, por ser a funcionalidade central do sistema. As demais funcionalidades, como recomendações, recursos sociais e análise de dados, são planejadas para etapas posteriores.

Para auxiliar na organização das etapas de desenvolvimento, foi utilizado o *GitHub Projects*, no qual as tarefas foram separadas, categorizadas e acompanhadas pela equipe.

4. Requisitos

Nesta seção são apresentados os requisitos do *Salsilauncher*, divididos em requisitos funcionais e não funcionais. Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer, enquanto os requisitos não funcionais definem restrições e características de qualidade do sistema.

4.1. Requisitos Funcionais

- RF01: O sistema deve permitir o cadastro manual de jogos.
- RF02: O sistema deve permitir a edição das informações de jogos cadastrados;
- RF03: O sistema deve permitir a remoção de jogos da biblioteca do usuário;
- RF04: O sistema deve exibir uma biblioteca contendo os jogos cadastrados pelo usuário;
- RF05: O sistema deve permitir a pesquisa de jogos por meio de filtros e palavras-chave;
- RF06: O sistema deve permitir a execução de jogos diretamente pela aplicação;
- RF07: O sistema deve registrar automaticamente o tempo de utilização dos jogos executados pelo *Salsilauncher*;
- RF08: O sistema deve armazenar o histórico de utilização dos jogos cadastrados;
- RF09: O sistema deve obter automaticamente metadados de jogos por meio de integrações com serviços externos;
- RF10: O sistema deve permitir a vinculação de contas de plataformas externas compatíveis;
- RF11: O sistema deve permitir a sincronização de bibliotecas de jogos provenientes de plataformas externas;
- RF12: O sistema deve permitir que usuários registrem avaliações para jogos cadastrados;
- RF13: O sistema deve gerar recomendações de jogos com base no histórico de utilização e avaliações dos usuários;
- RF14: O sistema deve disponibilizar informações analíticas relacionadas ao perfil de utilização do usuário;
- RF15: O sistema deve permitir o cadastro e autenticação de usuários;
- RF16: O sistema deve permitir o gerenciamento de amizades entre usuários;
- RF17: O sistema deve permitir a visualização de informações públicas de usuários adicionados como amigos;
- RF18: O sistema deve exportar e importar as informações da biblioteca do usuário, para que o mesmo a acesse de outros dispositivos;
- RF19: O sistema deve permitir a criação, edição e exclusão de coleções personalizadas de jogos na biblioteca do usuário;
- RF20: O sistema deve permitir a visualização e edição das informações de perfil do usuário;

4.2. Requisitos Não Funcionais

- RNF01: O sistema deve apresentar uma interface gráfica intuitiva e de fácil utilização;
- RNF02: O sistema deve ser executado como aplicação *desktop*;
- RNF03: O sistema deve ser compatível com o sistema operacional Windows;
- RNF04: O sistema deve possuir arquitetura que possibilite futura compatibilidade com sistemas Linux;
- RNF05: O sistema deve garantir a persistência dos dados dos usuários e jogos cadastrados;
- RNF06: O sistema deve garantir a integridade e consistência dos dados armazenados;
- RNF07: O sistema deve apresentar tempo de resposta adequado para operações de consulta e pesquisa;
- RNF08: O sistema deve proteger informações de autenticação dos usuários;
- RNF09: O sistema deve permitir integração com serviços externos de forma segura;
- RNF10: O sistema deve ser desenvolvido de forma modular, permitindo manutenção e expansão das funcionalidades;
- RNF11: O sistema deve suportar a adição de novas integrações com plataformas de jogos sem necessidade de reformulação completa da arquitetura;

5. Modelagem

Esta seção apresenta a modelagem inicial do *Salsilauncher*, com foco nos casos de uso identificados para a aplicação. A modelagem foi dividida em três grupos principais: biblioteca de jogos, integração e recomendação, e funcionalidades sociais com *dashboard* analítico.

5.1. Casos de Uso

Os casos de uso representam as principais interações entre os usuários e o sistema. Eles descrevem as funcionalidades previstas para o *Salsilauncher*, indicando quais ações podem ser executadas pelo usuário e quais integrações externas são utilizadas pela aplicação.

5.1.1 Biblioteca

O diagrama da Figura 1 apresenta os casos de uso relacionados ao módulo de biblioteca de jogos. Esse módulo concentra as funcionalidades centrais do *Salsilauncher*, como cadastro, edição, remoção, pesquisa, execução de jogos, visualização de tempo de uso, gerenciamento de coleções e importação de dados de perfil.

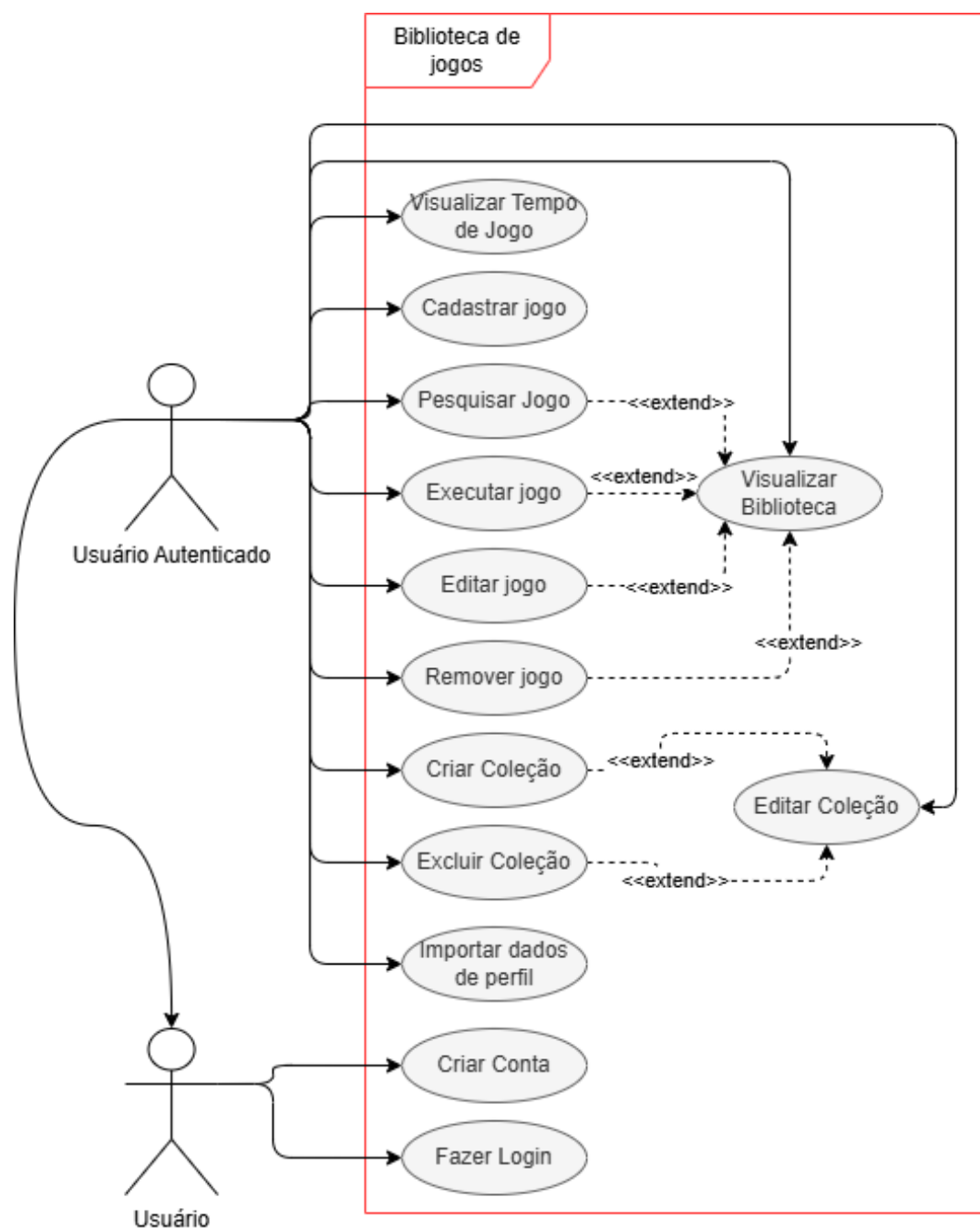


Figura 1 – Diagrama de casos de uso da Biblioteca de Jogos

5.1.2 Integração e Recomendação

O diagrama da Figura 2 apresenta os casos de uso relacionados às funcionalidades de integração com serviços externos e ao sistema de recomendação. Nesse grupo estão incluídas a importação da biblioteca da *Steam*, a avaliação de jogos e a visualização de recomendações personalizadas.

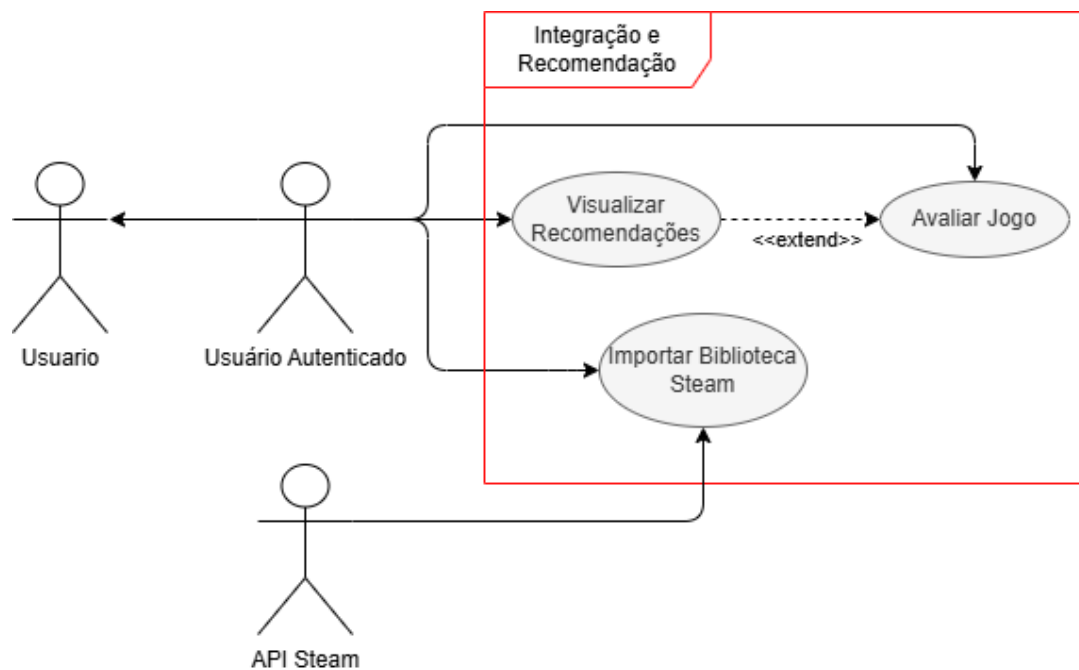


Figura 2 – Diagrama de casos de uso de Integração e Recomendação

5.1.3 Social e *Dashboard*

A Figura 3 apresenta os casos de uso relacionados às funcionalidades sociais e ao *dashboard* analítico. Essas funcionalidades permitem que o usuário visualize perfis, gerencie amizades, consulte atividades de amigos e acesse informações analíticas sobre seu uso da aplicação.

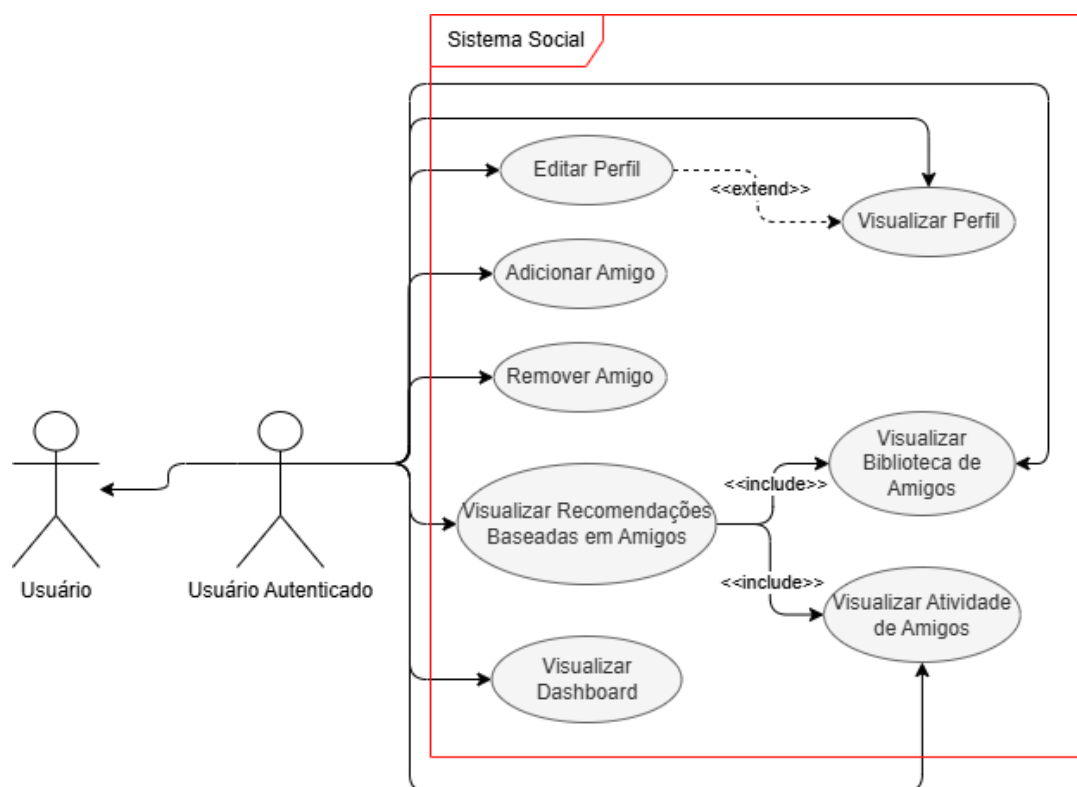


Figura 3 – Diagrama de casos de uso do sistema Social e *Dashboard* Analítico

As especificações completas dos casos de uso foram organizadas no Apêndice A. Como exemplo, a Tabela 1 apresenta a especificação do caso de uso de cadastro de jogo, uma das funcionalidades centrais da biblioteca.

Tabela 1 – Caso de Uso UC-03 - Cadastrar Jogo

Objetivo	Permitir que o usuário adicione manualmente um jogo à sua biblioteca.
RF/RNF Relacionados	RF01, RF09, RNF05
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Alta
Criticalidade	Alta
Condição de Entrada	Usuário seleciona a opção de cadastrar jogo.
Fluxo Principal	1. O usuário acessa a tela de cadastro de jogo. 2. O sistema solicita as informações do jogo. 3. O usuário informa os dados necessários, incluindo o caminho do executável. 4. O sistema valida as informações fornecidas. 5. O sistema busca metadados automaticamente, quando possível. 6. O sistema adiciona o jogo à biblioteca do usuário.
Fluxo Alternativo	Caso o caminho do executável seja inválido ou inexistente, o sistema exibe uma mensagem de erro e cancela o cadastro.
Pós-condições	O jogo é armazenado e passa a ser exibido na biblioteca do usuário.
Regras de Negócio	O caminho do executável deve ser válido e acessível pelo sistema.

5.2. Arquitetura do Motor de Recomendação Baseado em Pesos e Tags

O sistema de recomendação projetado para o *Salsilauncher* opera segundo uma abordagem de filtragem baseada em conteúdo, estruturada a partir do mapeamento de pesos e categorias (*tags*). O mecanismo visa identificar o perfil de interesse individual de cada usuário sem necessitar da exposição explícita de formulários de preferência, utilizando métricas comportamentais acumuladas.

5.2.1 Perfil Oculto do Usuário e Fatores de Preferência

Cada usuário cadastrado possui um perfil numérico implícito (oculto) mantido no banco de dados, que engloba o universo das *tags* de jogos registradas na plataforma. Para cada *tag* t , associa-se um fator numérico $F(t) \in R$, que atua como um coeficiente de afinidade. A variação desses fatores ocorre de forma dinâmica a partir das interações do usuário na aplicação:

- **Tempo de Jogo (Incremento Principal):** A execução de títulos via *launcher* constitui o indicador primário para o ajuste das preferências. Ao término de cada sessão de jogo, o tempo de utilização registrado é contabilizado e convertido em um acréscimo proporcional aos fatores das *tags* associadas ao jogo executado.
- **Avaliações (Reviews):** A atribuição de avaliações quantitativas pelo usuário (em uma escala de 1 a 5) impacta diretamente os pesos das *tags* do título avaliado. Notas elevadas (4 e 5) aplicam incrementos positivos aos fatores, enquanto avaliações neutras ou baixas (1 a 3) resultam em decrementos ou ajustes neutros nos fatores correspondentes.

5.2.2 Algoritmo de Seleção e Relaxamento de Filtros

Para a geração da lista final de recomendações a ser exibida na interface, o sistema executa um algoritmo de busca iterativa baseado no ordenamento dos fatores de preferência do usuário. O processo segue os seguintes passos:

1. **Ordenamento do Perfil:** O sistema recupera os fatores $F(t)$ do usuário e ordena as *tags* em ordem decrescente de pontuação, selecionando um conjunto inicial com as N *tags* de maior peso (por exemplo, um conjunto de $N = 5$ *tags* prioritárias: Ação, RPG, Multiplayer, Puzzle e Aventura).
2. **Busca Intersecção Iterativa:** O motor realiza uma consulta no banco de dados buscando por jogos que contenham simultaneamente a intersecção de todas as N *tags* do conjunto atual.
3. **Relaxamento Progressivo do Filtro (Range Reduction):** Caso a quantidade de jogos retornados pela consulta seja insuficiente para preencher a cota mínima estabelecida para a exibição de recomendações, o algoritmo reduz a restrição da busca decrementando o tamanho do conjunto em 1 ($N \leftarrow N - 1$). A busca passa, então, a consultar jogos que contemplem as $N - 1$ *tags* principais do usuário.
4. **Conclusão do Ciclo:** O ciclo de consulta e decremento do intervalo de busca é repetido iterativamente até que a quantidade desejada de jogos recomendados seja atingida.

6. Protótipo

O protótipo atual do *Salsilauncher* foi desenvolvido com o objetivo de representar visualmente a estrutura inicial da aplicação e validar a organização das principais telas previstas para o sistema. Nesta etapa, o protótipo contempla principalmente as funcionalidades relacionadas à autenticação de usuários, navegação pela interface, visualização da biblioteca e cadastro de jogos.

As telas foram elaboradas com foco em uma interface escura, utilizando elementos visuais em vermelho para destacar ações principais e áreas selecionadas. Essa identidade visual busca manter consistência entre as telas e facilitar a identificação das funcionalidades disponíveis ao usuário.

O protótipo inclui telas de login, criação de conta, página inicial da biblioteca, menu lateral, área de coleções, cadastro manual de jogos e escaneamento de pastas. Essas telas servem como referência para o desenvolvimento do *frontend* e para a validação inicial da experiência de uso.

As imagens completas do protótipo estão disponíveis no Apêndice B

7. Plano de Testes

O plano de testes tem como objetivo verificar se as funcionalidades do *Salsilauncher* atendem aos requisitos especificados para o sistema. Os testes foram definidos com base nos casos de uso e nas funcionalidades previstas para a aplicação, contemplando tanto requisitos funcionais quanto requisitos não funcionais.

Os testes do *Salsilauncher* serão realizados de forma incremental ao longo do desenvolvimento. A validação das funcionalidades será conduzida por meio de testes automatizados e testes funcionais.

Os testes automatizados serão divididos em duas frentes principais:

- **Backend (FastAPI e SQLAlchemy):** utilização do *framework Pytest* (pytest, 2026) para validação de *endpoints*, regras de negócio e integração do sistema. Serão empregadas técnicas de *mocking* para simular APIs externas e operações de banco de dados durante a esteira de CI/CD.
- **Frontend (React + TypeScript):** utilização de *Jest* e *React Testing Library* (Jest, 2026; Testing Library, 2026) para testes unitários e de componentes, além de *Playwright* ou *Cypress* (Microsoft, 2026a; Cypress, 2026) para testes *End-to-End*.

Os testes serão executados continuamente durante o desenvolvimento, garantindo a detecção precoce de falhas e a prevenção de regressões. As tabelas completas dos casos de teste estão disponíveis no Apêndice C.

A Tabela 2 apresenta um exemplo de caso de teste voltado ao cadastro de jogos, funcionalidade essencial para o funcionamento da biblioteca do *Salsilauncher*.

Tabela 2 – Caso de teste CT - 01

Identificador	CT - 01
Funcionalidade	Cadastro de jogo
Objetivo	Verificar se um jogo pode ser adicionado à biblioteca.
Pré-condição	Nenhuma.
Entrada	Nome do jogo e caminho válido do executável.
Resultado esperado	O jogo deve ser salvo e exibido na biblioteca do usuário.
Exceção (Caminho Triste)	Se o caminho do executável for inválido ou inexistente, o sistema deve barrar o cadastro e exibir uma mensagem de erro ("Caminho do arquivo não encontrado").

8. Resultados e Discussões

Neste capítulo são apresentados e discutidos os resultados obtidos até o estágio atual de desenvolvimento do *Salsilauncher*, correlacionando os artefatos produzidos (arquitetura, protótipos e testes) com os objetivos estabelecidos para o projeto. Como a metodologia adotada prevê o desenvolvimento incremental, as discussões concentram-se no núcleo funcional da biblioteca de jogos e na validação da infraestrutura cliente-servidor.

8.1. Estágio do Desenvolvimento Incremental

O desenvolvimento do *Salsilauncher* priorizou a construção da biblioteca unificada de jogos como o módulo central do sistema. A implementação deste núcleo viabilizou a execução das funcionalidades essenciais de cadastro manual (RF01), edição (RF02), remoção (RF03) e visualização de jogos (RF04).

A abordagem incremental permitiu isolar o comportamento da aplicação em ambiente *desktop* antes da integração completa com serviços externos. Observou-se que o gerenciamento local de arquivos executáveis atendeu às restrições de persistência (RNF05) e integridade de dados (RNF06), permitindo que o *launcher* inicialize jogos do sistema operacional e monitore o tempo de utilização de forma automática (RF06 e RF07).

8.2. Serviços no Backend com FastAPI e SQLAlchemy

No *backend*, foram construídos os *endpoints* da API REST utilizando o *framework FastAPI* em conjunto com o *SQLModel* para interação com o banco de dados *PostgreSQL*. A estrutura implementada processa requisições fundamentais, tais como a criação e vinculação de jogos ao perfil do usuário (POST /) e a atualização dos dados cadastrais do jogo (PUT /game_id). As regras de negócio estabelecidas garantem a persistência e a integridade do histórico individual de cada usuário.

8.3. Interface Gráfica e Integração Frontend

A aplicação cliente, desenvolvida em *React* com *TypeScript* teve seus componentes integrados à API do *backend*. As interfaces de usuário, que incluem as telas de edição de informações do jogo e gerenciamento de coleções personalizadas, foram projetadas para fornecer retorno visual durante o carregamento dos dados e permitir a filtragem dinâmica de títulos no lado do cliente.

A integração entre o *frontend* e o *backend* manteve-se estável durante a realização das operações essenciais da biblioteca, atendendo às exigências de usabilidade (RNF01) e de persistência (RNF05).

8.3.1 Isolamento de Credenciais da API Externa

Centralizar as requisições direcionadas à *Steam Web API* no Salsilauncher Server protegeu a chave de API (*API key*) da aplicação, eliminando a necessidade do usuário final de possuir ou fornecer credenciais de desenvolvedor (RNF08 e RNF09).

8.3.2 Eficiência da Pipeline de Metadados e Caching

A estratégia de busca em *pipeline* implementada no servidor central otimizou a consulta de informações de jogos. Ao realizar a busca primária no banco de dados *PostgreSQL* local antes de acionar a *Steam Web API*, o sistema reduziu a latência no retorno dos dados e evitou o bloqueio por limite de requisições (*rate limiting*). Os testes preliminares demonstraram que a persistência automática dos metadados consultados transforma requisições externas subsequentes em consultas locais de alta velocidade, atendendo ao tempo de resposta adequado estipulado no RNF07.

8.4. Avaliação das Interfaces e Prototipagem

A prototipagem da interface gráfica desenvolvida em *React* e *Electron* permitiu validar o fluxo de navegação do usuário antes do término do desenvolvimento do *frontend*. A disposição dos elementos no menu lateral e a organização em grade dos títulos cadastrados demonstraram alinhamento com a diretriz de interface intuitiva e de fácil utilização (RNF01).

Conforme detalhado no Apêndice B, as telas de escaneamento de diretórios locais e gerenciamento de coleções personalizadas foram projetadas para minimizar a quantidade de cliques necessários para a inclusão de jogos na biblioteca, validando o requisito RF19 na etapa de concepção gráfica.

8.5. Análise dos Resultados dos Testes

A validação do sistema foi conduzida com base no Plano de Testes estruturado no tópico 7. Os testes foram aplicados sobre as funcionalidades já implementadas e sobre os serviços do *backend*.

8.5.1 Testes Funcionais e de Execução

Os testes voltados ao gerenciamento da biblioteca (Tabela 27 a Tabela 30) apresentaram comportamento conforme o esperado, garantindo a validação de formulários com caminhos de executáveis inválidos e a filtragem por palavras-chave na barra de pesquisa.

Nos testes de execução e monitoramento (Tabela 31 a Tabela 33), o sistema registrou corretamente o processo do jogo no sistema operacional Windows (RNF03), contabilizando o tempo decorrido no perfil do usuário e bloqueando tentativas de execução duplicada de uma mesma aplicação.

8.5.2 Testes de Desempenho e Integração

A avaliação do *endpoint* de estatísticas e consultas analíticas (Tabela 45) confirmou que o tempo de resposta do *backend* em *FastAPI* manteve-se abaixo do limite estipulado de 1,5 segundos em ambiente de teste com carga média de dados. A integração com o banco manteve a consistência nas operações de leitura e escrita do histórico de uso do usuário.

9. Considerações Finais

Este trabalho apresentou o desenvolvimento e a modelagem do *Salsilauncher*, uma aplicação *desktop* concebida para mitigar a fragmentação das bibliotecas de jogos digitais decorrente da dispersão de títulos entre múltiplas plataformas de distribuição e executáveis locais. A proposta buscou centralizar a gestão e a execução de jogos em uma interface unificada, estabelecendo uma arquitetura sólida e extensível para o acompanhamento de métricas de utilização e integração com serviços externos.

9.1. Síntese das Entregas e Objetivos Alcançados

Em conformidade com a metodologia de desenvolvimento incremental adotada para o projeto, as entregas desta etapa concentraram-se na consolidação da Fase 1, correspondente ao núcleo funcional da biblioteca de jogos. Os objetivos propostos para o módulo principal foram devidamente atingidos com a implementação dos serviços de *backend* em *FastAPI* e da interface em *React* e *Electron*.

A aplicação demonstrou capacidade operacional para a realização do cadastro manual (RF01), edição (RF02), remoção (RF03) e consulta filtrada de jogos (RF05). Além disso, a integração com o sistema operacional Windows permitiu a execução direta dos títulos a partir do *launcher* (RF06) e a contagem automática do tempo de utilização (RF07) mantida sob persistência de dados relacional via *SQLModel* e *PostgreSQL* (RNF05).

A validação da arquitetura cliente-servidor por meio do *Salsilauncher Server* confirmou a viabilidade técnica da mediação centralizada para requisições externas. Essa abordagem isolou a chave de acesso da *Steam Web API* no ambiente do servidor (RNF08 e RNF09) e viabilizou uma busca em *pipeline* com *caching* de metadados, reduzindo chamadas redundantes à API e assegurando tempo de resposta adequado para as consultas dos usuários (RNF07).

9.2. Limitações Identificadas

Por se tratar de um projeto em estágio contínuo de desenvolvimento, identificam-se limitações decorrentes do delimitamento de escopo estabelecido para a primeira fase:

- **Restrição de Plataforma Operacional:** A aplicação cliente encontra-se homologada e testada prioritariamente para o sistema operacional Windows, necessitando de ajustes de compatibilidade para suporte nativo a ambientes *Linux* (RNF04), que seria possível via empacotamento *Electron*;
- **Escopo de Integração Externa:** A *pipeline* de metadados e a sincronização automática de bibliotecas estão restritas à *Steam Web API*, dependendo de cadastros manuais para títulos originados de outras plataformas;
- **Estágio dos Módulos Avançados:** Os recursos voltados ao sistema social de amigos (RF16 e RF17), ao algoritmo de recomendação personalizada (RF13) e ao *dashboard* analítico de utilização (RF14) permanecem em nível de modelagem e prototipagem, aguardando a consolidação final da base de dados dos usuários para implementação em código.

9.3. Trabalhos Futuros

Como continuidade ao desenvolvimento do *Salsilauncher*, recomendam-se as seguintes etapas e trabalhos futuros:

1. **Implementação do Sistema Social e Comunicação:** Construção das rotas do servidor central para o gerenciamento de solicitações de amizade, visualização de atividades em tempo real e compartilhamento de bibliotecas entre usuários vinculados, evoluindo futuramente para módulos de comunicação via chat de texto e voz;
2. **Desenvolvimento do Motor de Recomendação:** Implementação de algoritmos de filtragem colaborativa e baseada em conteúdo, utilizando as avaliações cadastradas (RF12) e o histórico de uso dos usuários e de suas amizades como insumo analítico;
3. **Construção do Dashboard Analítico:** Desenvolvimento de componentes gráficos interativos na interface do usuário para exibição estatística detalhada do perfil de consumo e preferências de jogos;
4. **Expansão de Integrações e Portabilidade:** Inclusão de novas integrações modulares na *pipeline* do servidor (como *Epic Games* e *GOG*) e adaptação do empacotamento em *Electron* para garantir suporte multiplataforma com distribuições *Linux*.

Referências

- Cypress. *Cypress Documentation*. 2026. <<https://docs.cypress.io/>>. Acesso em: 09 junho 2026. Citado na página 11.
- Discord. *Discord Support*. 2026. <<https://support.discord.com/>>. Acesso em: 03 junho 2026. Citado na página 4.
- FastAPI. *FastAPI Documentation*. 2026. Acesso em: 13 maio 2026. Disponível em: <<https://fastapi.tiangolo.com/>>. Citado na página 3.
- Git. *Git Reference*. 2026. Acesso em: 22 maio 2026. Disponível em: <<https://git-scm.com/docs>>. Citado na página 4.
- GitHub. *About Projects*. 2026. Acesso em: 22 maio 2026. Disponível em: <<https://docs.github.com/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>>. Citado na página 4.
- GitHub. *GitHub Docs*. 2026. <<https://docs.github.com/>>. Acesso em: 29 maio 2026. Citado na página 4.
- Google. *Ajuda do Apresentações Google*. 2026. <<https://support.google.com/docs/topic/9054003?hl=pt-BR>>. Acesso em: 03 junho 2026. Citado na página 5.
- Jest. *Jest Documentation*. 2026. <<https://jestjs.io/docs/getting-started>>. Acesso em: 06 junho 2026. Citado na página 11.
- MARINHO, L. H. et al. A recommender system in python. 2015. Acesso em: 4 abril 2026. Disponível em: <<https://www.academia.edu/download/72929413/331-1.pdf>>. Citado nas páginas 1 e 2.
- Meta Open Source. *React Documentation*. 2026. Acesso em: 13 maio 2026. Disponível em: <<https://react.dev/>>. Citado na página 3.
- Microsoft. *Playwright Documentation*. 2026. <<https://playwright.dev/>>. Acesso em: 09 junho 2026. Citado na página 11.
- Microsoft. *TypeScript Documentation*. 2026. Acesso em: 15 maio 2026. Disponível em: <<https://www.typescriptlang.org/docs/>>. Citado na página 3.
- Microsoft. *Visual Studio Code Documentation*. 2026. <<https://code.visualstudio.com/docs>>. Acesso em: 29 maio 2026. Citado na página 4.
- Newzoo. *Global Gamer Study 2024*. 2024. <<https://newzoo.com/global-gamer-study>>. Pesquisas globais sobre o comportamento gamer. Acesso em: 02 abril 2026. Citado na página 1.
- OpenJS Foundation. *Electron Documentation*. 2026. Acesso em: 15 maio 2026. Disponível em: <<https://www.electronjs.org/docs/latest>>. Citado na página 3.
- Overleaf. *Overleaf Documentation*. 2026. <<https://docs.overleaf.com/>>. Acesso em: 03 junho 2026. Citado na página 5.
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2026. Acesso em: 20 maio 2026. Disponível em: <<https://www.postgresql.org/docs/>>. Citado na página 3.
- pytest. *pytest documentation*. 2026. <<https://docs.pytest.org/en/stable/>>. Acesso em: 06 junho 2026. Citado na página 11.
- Python Software Foundation. *Python Documentation*. 2026. <<https://docs.python.org/3/>>. Acesso em: 27 maio 2026. Citado na página 3.
- SCOTT, C.; LEWIS, B. Advancements in web application development: An analytical review of ruby on rails, python frameworks and cloud-centric solutions. 2024. Acesso em: 4 abril 2026. Disponível em: <<https://www.researchgate.net/publication/394759430>>. Citado na página 2.

SteamDB. *Steam Concurrent Players Statistics*. 2026. <<https://steamdb.info>>. Banco de informações sobre a plataforma Steam. Acesso em: 02 abril 2026. Citado na página 1.

TAKAHASHI, M. M. *Algoritmos de Recomendação*. 2014. <<https://bccdev.ime.usp.br/tccs/2014/marcost/index.html>>. Trabalho de Conclusão de Curso, Instituto de Matemática e Estatística da Universidade de São Paulo. Acesso em: 4 abril 2026. Citado na página 3.

Testing Library. *React Testing Library*. 2026. <<https://testing-library.com/docs/react-testing-library/intro/>>. Acesso em: 06 junho 2026. Citado na página 11.

Valve Corporation. *Steam Web API Documentation*. 2026. Acesso em: 20 maio 2026. Disponível em: <https://developer.valvesoftware.com/wiki/Steam_Web_API>. Citado na página 3.

APÊNDICE A – Tabelas de Casos de Uso

Este apêndice apresenta as tabelas completas de especificação dos casos de uso do *Salsilauncher*. As tabelas foram organizadas em sessões referentes ao seu escopo, e cada tabela descreve o objetivo do caso de uso, requisitos relacionados, atores envolvidos, prioridade, pré-condições, frequência de uso, criticidade, condição de entrada, fluxo principal, fluxo alternativo, pós-condições e regras de negócio.

1. Biblioteca

A [Tabela 3](#) apresenta a especificação do caso de uso UC-01, responsável por permitir o cadastro inicial de novos usuários na plataforma.

Tabela 3 – Caso de Uso UC-01 - Criar Conta

Objetivo	Permitir que um novo usuário realize seu cadastro na plataforma.
RF/RNF Relacionados	RF15, RNF08
Atores	Usuário
Prioridade	Alta
Pré-condições	Nenhuma.
Frequência de uso	Baixa
Criticidade	Alta
Condição de Entrada	Usuário seleciona a opção de cadastro.
Fluxo Principal	1. O usuário acessa a tela de cadastro. 2. O sistema solicita o preenchimento dos seguintes campos: • E-mail • Senha 3. O usuário informa os dados solicitados. 4. O sistema valida as informações. 5. O sistema cria a conta do usuário.
Fluxo Alternativo	Caso já exista uma conta associada ao e-mail informado, o sistema exibe uma mensagem de erro e cancela a operação.
Pós-condições	Uma nova conta é criada e armazenada no sistema.
Regras de Negócio	Cada usuário deve possuir um e-mail único cadastrado no sistema.

A [Tabela 4](#) detalha o caso de uso UC-02, referente ao procedimento de autenticação de usuários previamente cadastrados para acesso às funcionalidades restritas do sistema.

Tabela 4 – Caso de Uso UC-02 - Fazer Login

Objetivo	Permitir que um usuário cadastrado acesse sua conta na plataforma.
RF/RNF Relacionados	RF15, RNF08
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve possuir uma conta cadastrada.
Frequência de uso	Alta
Criticidade	Alta
Condição de Entrada	Usuário acessa a tela de <i>login</i> .

Fluxo Principal	1. O usuário acessa a tela de autenticação. 2. O sistema solicita o e-mail e a senha. 3. O usuário insere suas credenciais. 4. O sistema valida as informações. 5. O sistema concede acesso à plataforma.
Fluxo Alternativo	Caso as credenciais sejam inválidas, o sistema exibe uma mensagem de erro e solicita novamente os dados de acesso.
Pós-condições	O usuário é autenticado e redirecionado para a tela principal da aplicação.
Regras de Negócio	O sistema deve proteger as informações de autenticação dos usuários.

A [Tabela 5](#) especifica o caso de uso UC-03, destinado à inclusão manual de novos títulos na biblioteca do usuário.

Tabela 5 – Caso de Uso UC-03 - Cadastrar Jogo

Objetivo	Permitir que o usuário adicione manualmente um jogo à sua biblioteca.
RF/RNF Relacionados	RF01, RF09, RNF05
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado no sistema.
Frequência de uso	Média
Criticalidade	Alta
Condição de Entrada	Usuário seleciona a opção de cadastrar um novo jogo.
Fluxo Principal	1. O usuário acessa o formulário de cadastro de jogo. 2. O usuário informa os dados do jogo, incluindo título e diretório do arquivo executável. 3. O sistema valida os dados inseridos. 4. O sistema registra o jogo na biblioteca do usuário.
Fluxo Alternativo	O usuário pode fornecer o identificador de um serviço externo para buscar automaticamente os metadados do jogo.
Pós-condições	O jogo é cadastrado e exibido na biblioteca do usuário.
Regras de Negócio	O caminho do executável deve ser válido para viabilizar a posterior execução do jogo.

A [Tabela 6](#) apresenta as especificações do caso de uso UC-04, voltado à atualização e modificação das informações de jogos pertencentes à biblioteca.

Tabela 6 – Caso de Uso UC-04 - Editar Jogo

Objetivo	Permitir que o usuário modifique as informações de um jogo previamente cadastrado.
RF/RNF Relacionados	RF02, RNF05
Atores	Usuário
Prioridade	Média
Pré-condições	O jogo deve estar previamente cadastrado na biblioteca do usuário.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de edição nas informações de um jogo.
Fluxo Principal	1. O usuário seleciona um jogo cadastrado em sua biblioteca. 2. O sistema exibe o formulário de edição preenchido com os dados atuais do jogo. 3. O usuário altera os campos desejados. 4. O sistema valida as novas informações. 5. O sistema atualiza o registro do jogo.

Fluxo Alternativo	O usuário cancela a operação de edição e o sistema mantém as informações originais do jogo.
Pós-condições	As informações atualizadas do jogo são salvas e refletidas na biblioteca.
Regras de Negócio	Os dados alterados devem manter a consistência e a integridade da biblioteca do usuário.

A [Tabela 7](#) especifica o procedimento do caso de uso UC-05 para a exclusão de títulos da biblioteca do usuário.

Tabela 7 – Caso de Uso UC-05 - Remover Jogo

Objetivo	Permitir que o usuário remova um jogo de sua biblioteca.
RF/RNF Relacionados	RF03, RNF05
Atores	Usuário
Prioridade	Média
Pré-condições	O jogo deve estar cadastrado na biblioteca do usuário.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de remoção de um jogo.
Fluxo Principal	1. O usuário seleciona o jogo que deseja remover. 2. O sistema exibe uma mensagem solicitando confirmação da exclusão. 3. O usuário confirma a remoção. 4. O sistema exclui o jogo da biblioteca do usuário.
Fluxo Alternativo	Caso o usuário cancele a operação, o jogo permanece mantido na biblioteca sem alterações.
Pós-condições	O jogo é removido da biblioteca do usuário.
Regras de Negócio	A remoção do jogo da biblioteca não deve apagar arquivos do sistema operacional do usuário.

A [Tabela 8](#) detalha a estrutura do caso de uso UC-06, focado na busca e filtragem de títulos na biblioteca por meio de palavras-chave.

Tabela 8 – Caso de Uso UC-06 - Pesquisar Jogo

Objetivo	Permitir que o usuário busque jogos na biblioteca por palavras-chave ou filtros específicos.
RF/RNF Relacionados	RF05, RNF01, RNF07
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar na tela de visualização da biblioteca.
Frequência de uso	Alta
Criticalidade	Média
Condição de Entrada	Usuário insere um termo na barra de pesquisa ou aplica um filtro.
Fluxo Principal	1. O usuário digita o nome do jogo ou seleciona um filtro na interface. 2. O sistema consulta a biblioteca cadastrada do usuário. 3. O sistema exibe os jogos correspondentes aos critérios informados.
Fluxo Alternativo	Se nenhum jogo corresponder aos critérios informados, o sistema exibe uma mensagem indicando que nenhum resultado foi encontrado.
Pós-condições	A interface apresenta apenas os jogos que atendem aos filtros aplicados.
Regras de Negócio	O sistema deve apresentar tempo de resposta adequado em operações de consulta e pesquisa.

A [Tabela 9](#) apresenta a especificação do caso de uso UC-07, responsável pela exibição organizada dos jogos pertencentes ao usuário.

Tabela 9 – Caso de Uso UC-07 - Visualizar Biblioteca

Objetivo	Permitir que o usuário visualize a lista completa dos jogos cadastrados em sua biblioteca.
RF/RNF Relacionados	RF04, RNF01
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Alta
Criticalidade	Alta
Condição de Entrada	Usuário acessa a tela principal ou seleciona a opção de biblioteca.
Fluxo Principal	1. O usuário acessa a área de biblioteca do sistema. 2. O sistema recupera os jogos cadastrados do usuário. 3. O sistema exibe os jogos em formato de lista ou grade na interface gráfica.
Fluxo Alternativo	Caso a biblioteca não possua jogos cadastrados, o sistema exibe uma tela inicial amigável orientando o usuário a adicionar seus primeiros títulos.
Pós-condições	A biblioteca do usuário é exibida na interface.
Regras de Negócio	A interface gráfica deve ser intuitiva e de fácil visualização.

A Tabela 10 detalha os fluxos e condições do caso de uso UC-08, voltado à inicialização direta dos jogos a partir da aplicação.

Tabela 10 – Caso de Uso UC-08 - Executar Jogo

Objetivo	Permitir que o usuário inicie um jogo diretamente por meio da aplicação.
RF/RNF Relacionados	RF06, RF07, RNF02
Atores	Usuário
Prioridade	Alta
Pré-condições	O jogo selecionado deve possuir o caminho do executável válido cadastrado.
Frequência de uso	Alta
Criticalidade	Alta
Condição de Entrada	Usuário clica no botão de execução de um jogo.
Fluxo Principal	1. O usuário seleciona um jogo da biblioteca e clica em executar. 2. O sistema localiza o executável associado ao jogo. 3. O sistema inicia o processo do jogo no sistema operacional. 4. O sistema inicia o monitoramento e registro automático do tempo de uso.
Fluxo Alternativo	Se o arquivo executável do jogo não for localizado, o sistema exibe uma mensagem de erro informando que o caminho cadastrado é inválido.
Pós-condições	O jogo é executado e o tempo de sessão passa a ser contabilizado pelo sistema.
Regras de Negócio	O sistema deve registrar o tempo de uso de forma transparente durante a execução do processo.

A Tabela 11 especifica o caso de uso UC-09, referente ao acompanhamento do histórico e do tempo acumulado de utilização dos jogos.

Tabela 11 – Caso de Uso UC-09 - Visualizar Tempo de Jogo

Objetivo	Permitir que o usuário consulte o tempo total de utilização e o histórico de execuções de seus jogos.
RF/RNF Relacionados	RF07, RF08, RNF01
Atores	Usuário

Prioridade	Média
Pré-condições	O usuário deve ter executado ao menos um jogo via <i>launcher</i> .
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário acessa os detalhes de um jogo ou o painel estatístico.
Fluxo Principal	1. O usuário acessa a página de detalhes de um jogo ou área de estatísticas. 2. O sistema recupera os registros de utilização armazenados. 3. O sistema calcula e exibe o tempo total jogado e o histórico recente de uso.
Fluxo Alternativo	Caso o jogo nunca tenha sido executado pela aplicação, o sistema indica que o tempo jogado é zero.
Pós-condições	O usuário visualiza as informações sobre o tempo de utilização do jogo.
Regras de Negócio	O tempo armazenado no histórico deve ser mantido de forma persistente e consistente no banco de dados.

A [Tabela 12](#) apresenta o caso de uso UC-10, destinado ao agrupamento e organização personalizada dos jogos da biblioteca.

Tabela 12 – Caso de Uso UC-10 - Criar Coleção

Objetivo	Permitir que o usuário crie coleções personalizadas para agrupar e organizar seus jogos.
RF/RNF Relacionados	RF19, RNF01
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de criar nova coleção.
Fluxo Principal	1. O usuário solicita a criação de uma nova coleção. 2. O sistema solicita um nome e uma descrição opcional para a coleção. 3. O usuário insere as informações e confirma. 4. O sistema valida os dados e cria a coleção na conta do usuário.
Fluxo Alternativo	Se o nome fornecido para a coleção estiver em branco, o sistema solicita o preenchimento obrigatório do campo.
Pós-condições	Uma nova coleção é criada e disponibilizada na interface do usuário.
Regras de Negócio	Coleções permitem ao usuário agrupar títulos conforme categorias de sua preferência.

A [Tabela 13](#) detalha o fluxo do caso de uso UC-11, voltado à alteração de nome, descrição e inclusão de jogos em coleções existentes.

Tabela 13 – Caso de Uso UC-11 - Editar Coleção

Objetivo	Permitir a alteração do nome, descrição ou lista de jogos associados a uma coleção personalizada.
RF/RNF Relacionados	RF19, RNF05
Atores	Usuário
Prioridade	Média
Pré-condições	A coleção a ser editada deve existir na conta do usuário.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de editar uma coleção específica.

Fluxo Principal	1. O usuário seleciona a coleção desejada e aciona a opção de edição. 2. O sistema apresenta as informações atuais e a lista de jogos vinculados. 3. O usuário altera os dados cadastrais ou insere/remove jogos da coleção. 4. O sistema atualiza o registro da coleção.
Fluxo Alternativo	O usuário cancela as edições, mantendo a coleção em seu estado anterior.
Pós-condições	As alterações na coleção são salvas e refletidas na biblioteca.
Regras de Negócio	A alteração dos vínculos da coleção não afeta a presença dos jogos na biblioteca principal do usuário.

A [Tabela 14](#) especifica o procedimento do caso de uso UC-12 para a remoção de coleções organizacionais do usuário.

Tabela 14 – Caso de Uso UC-12 - Excluir Coleção

Objetivo	Permitir a exclusão de uma coleção personalizada da biblioteca.
RF/RNF Relacionados	RF19, RNF05
Atores	Usuário
Prioridade	Média
Pré-condições	A coleção selecionada deve existir na conta do usuário.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de excluir uma coleção.
Fluxo Principal	1. O usuário seleciona a coleção e solicita sua exclusão. 2. O sistema exibe uma mensagem solicitando a confirmação do procedimento. 3. O usuário confirma a remoção. 4. O sistema exclui o agrupamento da coleção.
Fluxo Alternativo	O usuário cancela a exclusão e a coleção é mantida na conta.
Pós-condições	A coleção é removida da conta do usuário.
Regras de Negócio	A exclusão de uma coleção não deve remover os jogos nela contidos da biblioteca geral do usuário.

A [Tabela 15](#) especifica o caso de uso UC-13, referente à portabilidade da biblioteca entre diferentes dispositivos.

Tabela 15 – Caso de Uso UC-13 - Importar Dados de Perfil

Objetivo	Permitir que o usuário exporte ou importe dados de sua biblioteca para sincronização entre dispositivos.
RF/RNF Relacionados	RF18, RNF05, RNF06
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve possuir um arquivo de dados válido gerado pela aplicação.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário aciona a funcionalidade de importação de dados de perfil nas configurações.
Fluxo Principal	1. O usuário acessa a opção de importação de dados. 2. O usuário seleciona o arquivo de dados em seu dispositivo. 3. O sistema lê e valida a estrutura e integridade do arquivo. 4. O sistema mescla ou atualiza a biblioteca do usuário com as novas informações.
Fluxo Alternativo	Caso o arquivo esteja corrompido ou em formato incompatível, o sistema cancela a importação e exibe uma mensagem de erro.

Pós-condições	A biblioteca do usuário é atualizada com base no arquivo importado.
Regras de Negócio	O sistema deve garantir a integridade e consistência de dados durante o processo de sincronização.

2. Integração e Recomendação

A [Tabela 16](#) apresenta o caso de uso UC-14, focado na submissão de avaliações e notas pelos usuários para os jogos cadastrados.

Tabela 16 – Caso de Uso UC-14 - Avaliar Jogo

Objetivo	Permitir que o usuário atribua notas ou avaliações aos jogos da sua biblioteca.
RF/RNF Relacionados	RF12, RNF05
Atores	Usuário
Prioridade	Média
Pré-condições	O jogo deve estar cadastrado na biblioteca do usuário.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário acessa a página de detalhes do jogo e seleciona a opção de avaliação.
Fluxo Principal	1. O usuário acessa a página do jogo que deseja avaliar. 2. O usuário escolhe a nota ou classificação desejada. 3. O sistema registra a avaliação associando-a ao perfil do usuário e ao jogo.
Fluxo Alternativo	Caso o jogo já possua uma avaliação prévia feita pelo usuário, o sistema permite alterar a nota existente.
Pós-condições	A avaliação é armazenada e poderá ser utilizada na geração de sugestões personalizadas.
Regras de Negócio	As avaliações registradas servem como insumo para os algoritmos de recomendação.

A [Tabela 17](#) especifica o caso de uso UC-15, focado na exibição de sugestões personalizadas de jogos geradas pelo sistema.

Tabela 17 – Caso de Uso UC-15 - Visualizar Recomendações

Objetivo	Apresentar ao usuário sugestões de jogos com base em seu perfil de uso e avaliações.
RF/RNF Relacionados	RF13, RNF01, RNF07
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve ter histórico de uso ou avaliações registradas no sistema.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário acessa a aba ou painel de recomendações da interface.
Fluxo Principal	1. O usuário acessa a área de recomendações. 2. O sistema recupera o perfil de uso e preferências do usuário. 3. O sistema processa o algoritmo de recomendação. 4. O sistema apresenta uma lista ordenada de sugestões de jogos ao usuário.
Fluxo Alternativo	Caso o usuário ainda não possua dados suficientes de uso, o sistema exibe recomendações genéricas ou populares na plataforma.

Pós-condições	O usuário visualiza títulos sugeridos adequados ao seu perfil de preferência.
Regras de Negócio	O cálculo de recomendação deve utilizar métricas de relevância e similaridade baseadas no histórico do usuário.

A [Tabela 18](#) detalha a estrutura do caso de uso UC-16, voltado à sincronização com a plataforma externa *Steam*.

Tabela 18 – Caso de Uso UC-16 - Importar Biblioteca Steam

Objetivo	Permitir a sincronização automática dos jogos pertencentes à conta do usuário na plataforma <i>Steam</i> .
RF/RNF Relacionados	RF10, RF11, RNF09
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve possuir uma conta na <i>Steam</i> pública ou com permissão de acesso via API.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário escolhe vincular e importar sua conta da <i>Steam</i> nas configurações da aplicação.
Fluxo Principal	1. O usuário informa seu identificador da <i>Steam</i> no sistema. 2. O sistema realiza uma requisição segura à API da plataforma externa. 3. O sistema obtém a lista de jogos vinculados à conta do usuário. 4. O sistema cadastra automaticamente os novos títulos na biblioteca do <i>Salsilauncher</i> .
Fluxo Alternativo	Se a conta informada for privada ou o identificador for inválido, o sistema exibe uma mensagem orientando o ajuste de privacidade na plataforma externa.
Pós-condições	A biblioteca do usuário é sincronizada com os jogos importados da plataforma externa.
Regras de Negócio	As integrações com serviços externos devem ser efetuadas de forma segura.

A [Tabela 19](#) detalha o caso de uso UC-17, referente à geração de recomendações derivadas do perfil de uso das amizades do usuário.

Tabela 19 – Caso de Uso UC-17 - Visualizar Recomendações Baseadas em Amigos

Objetivo	Exibir sugestões de jogos calculadas a partir das bibliotecas e preferências das amizades do usuário.
RF/RNF Relacionados	RF13, RF17, RNF01
Atores	Usuário
Prioridade	Baixa
Pré-condições	O usuário deve possuir amigos adicionados no sistema que compartilhem dados públicos.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário acessa o painel de recomendações sociais.

Fluxo Principal	1. O usuário aciona a opção de sugestões baseadas em amigos. 2. O sistema identifica as contas de amigos associadas ao usuário. 3. O sistema analisa os jogos mais jogados e melhor avaliados por esses contatos. 4. O sistema gera e exibe a lista de recomendações sociais na interface.
Fluxo Alternativo	Caso os amigos adicionados não possuam dados públicos compartilhados, o sistema exibe um aviso sobre a ausência de histórico suficiente.
Pós-condições	O usuário visualiza os jogos populares e recomendados no círculo de amigos.
Regras de Negócio	Somente dados públicos de perfil de usuários adicionados como amigos podem integrar a análise de recomendação social.

3. Funcionalidades Sociais e Dashboard Analítico

A [Tabela 20](#) especifica o caso de uso UC-18, focado na apresentação das informações cadastrais e públicas do usuário no sistema.

Tabela 20 – Caso de Uso UC-18 - Visualizar Perfil

Objetivo	Permitir que o usuário consulte as informações do seu perfil ou de outros usuários públicos.
RF/RNF Relacionados	RF17, RF20, RNF01
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado para visualizar dados de perfis no sistema.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário seleciona a opção de visualizar perfil na interface.
Fluxo Principal	1. O usuário acessa a área de perfil. 2. O sistema busca os dados cadastrais, resumo de biblioteca e atividades do perfil selecionado. 3. O sistema exibe as informações organizadas na interface gráfica.
Fluxo Alternativo	Se o perfil buscado for privado e não houver vínculo de amizade, o sistema exibe apenas as informações públicas restritas.
Pós-condições	As informações do perfil são exibidas na tela.
Regras de Negócio	Dados privados de usuários não autenticados ou não autorizados não devem ser expostos na interface.

A [Tabela 21](#) apresenta as especificações do caso de uso UC-19, voltado à atualização das informações pessoais e preferências de exibição do usuário.

Tabela 21 – Caso de Uso UC-19 - Editar Perfil

Objetivo	Permitir que o usuário modifique seus dados cadastrais, imagem de exibição e preferências de perfil.
RF/RNF Relacionados	RF20, RNF05, RNF08
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar devidamente autenticado na aplicação.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário acessa as configurações e escolhe a opção de editar perfil.

Fluxo Principal	1. O usuário acessa a tela de edição de perfil. 2. O sistema exibe os campos editáveis (nome de exibição, avatar e preferências). 3. O usuário altera os dados desejados e clica em salvar. 4. O sistema valida os campos e atualiza o perfil no banco de dados.
Fluxo Alternativo	Se o nome de exibição fornecido violar as regras de validação do sistema, uma mensagem de erro é exibida impedindo a alteração.
Pós-condições	As alterações cadastrais do perfil são persistidas no sistema.
Regras de Negócio	As informações de autenticação alteradas devem ser validadas e armazenadas de forma segura.

A [Tabela 22](#) especifica o procedimento do caso de uso UC-20 para o envio e aceitação de solicitações de amizade entre usuários da plataforma.

Tabela 22 – Caso de Uso UC-20 - Adicionar Amigo

Objetivo	Permitir o envio de solicitações de amizade para outros usuários cadastrados na aplicação.
RF/RNF Relacionados	RF16, RNF01
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário remetente e o usuário destinatário devem possuir contas ativas.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário busca por outro perfil e aciona o botão de adicionar amigo.
Fluxo Principal	1. O usuário pesquisa o nome de exibição de outro usuário no sistema. 2. O usuário clica na opção de enviar solicitação de amizade. 3. O sistema encaminha a solicitação ao destinatário. 4. Ao ser aceita pelo destinatário, o sistema estabelece o vínculo de amizade entre as contas.
Fluxo Alternativo	Caso a solicitação já tenha sido enviada ou os usuários já sejam amigos, o sistema impede o envio e exibe um aviso na tela.
Pós-condições	O vínculo de amizade é registrado e mantido no sistema.
Regras de Negócio	O gerenciamento de amizades é pré-requisito para o compartilhamento de bibliotecas e atividades.

A [Tabela 23](#) detalha a estrutura do caso de uso UC-21, responsável pelo desfazimento do vínculo de amizade mantido entre dois usuários.

Tabela 23 – Caso de Uso UC-21 - Remover Amigo

Objetivo	Permitir o encerramento do vínculo de amizade com outro usuário cadastrado.
RF/RNF Relacionados	RF16, RNF05
Atores	Usuário
Prioridade	Baixa
Pré-condições	Deve existir um vínculo de amizade prévio entre os dois usuários.
Frequência de uso	Baixa
Criticalidade	Baixa
Condição de Entrada	Usuário seleciona um contato na sua lista de amigos e escolhe remover amigo.

Fluxo Principal	1. O usuário acessa sua lista de amigos cadastrada. 2. O usuário escolhe o amigo que deseja remover e aciona a opção correspondente. 3. O sistema pede confirmação da remoção. 4. O usuário confirma e o sistema remove o vínculo entre as duas contas.
Fluxo Alternativo	O usuário cancela a confirmação de exclusão e o amigo permanece mantido na lista.
Pós-condições	O vínculo de amizade é excluído do banco de dados.
Regras de Negócio	Com a remoção da amizade, o acesso visual às atividades privadas e bibliotecas do usuário em questão é encerrado.

A [Tabela 24](#) detalha o caso de uso UC-22, destinado ao acompanhamento das ações e sessões de jogos recentes de usuários conectados.

Tabela 24 – Caso de Uso UC-22 - Visualizar Atividade de Amigos

Objetivo	Permitir a consulta às atividades recentes e jogos executados pelos amigos vinculados ao usuário.
RF/RNF Relacionados	RF17, RNF01
Atores	Usuário
Prioridade	Baixa
Pré-condições	O usuário deve possuir amigos adicionados que tenham gerado dados recentes de uso.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário acessa o painel social ou aba de atividades da comunidade.
Fluxo Principal	1. O usuário entra na área social da aplicação. 2. O sistema busca as últimas sessões de jogo e atualizações realizadas pelos seus amigos. 3. O sistema ordena e apresenta a linha do tempo de atividades recentes dos contatos.
Fluxo Alternativo	Se nenhum amigo tiver registrado atividades recentemente, o sistema exibe a mensagem de que não há atualizações disponíveis.
Pós-condições	A lista de atividades recentes da rede de amigos é exibida na interface.
Regras de Negócio	Somente atividades de contas associadas como amigas e autorizadas podem ser listadas.

A [Tabela 25](#) especifica o caso de uso UC-23, referente ao acesso e consulta aos jogos pertencentes a outros usuários adicionados.

Tabela 25 – Caso de Uso UC-23 - Visualizar Biblioteca de Amigos

Objetivo	Permitir que o usuário consulte a coleção de jogos cadastrada na conta de um amigo.
RF/RNF Relacionados	RF17, RNF01
Atores	Usuário
Prioridade	Baixa
Pré-condições	O usuário analisado deve ser amigo do usuário solicitante e possuir biblioteca pública/visível.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário acessa o perfil de um amigo e seleciona a opção de ver biblioteca.

Fluxo Principal	1. O usuário seleciona um amigo da sua lista de contatos. 2. O usuário clica na opção para visualizar a biblioteca do amigo selecionado. 3. O sistema busca os títulos públicos vinculados à conta do amigo. 4. O sistema apresenta a lista de jogos na interface gráfica.
Fluxo Alternativo	Caso a biblioteca do amigo esteja configurada como privada, o sistema bloqueia a exibição e apresenta uma mensagem explicativa.
Pós-condições	A biblioteca do amigo é apresentada para consulta em modo de leitura.
Regras de Negócio	A visualização de bibliotecas alheias é estritamente restrita à consulta, sem permissões de alteração.

A Tabela 26 apresenta a especificação do caso de uso UC-24, responsável por exibir informações analíticas sobre o perfil de utilização do usuário.

Tabela 26 – Caso de Uso UC-24 - Visualizar Dashboard

Objetivo	Permitir que o usuário visualize informações analíticas sobre sua utilização da plataforma.
RF/RNF Relacionados	RF14, RNF01, RNF07
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e possuir dados de utilização registrados.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário acessa a área de <i>dashboard</i> .
Fluxo Principal	1. O usuário acessa a tela de <i>dashboard</i>. 2. O sistema recupera os dados de utilização do usuário. 3. O sistema calcula informações analíticas, como tempo de jogo, jogos mais utilizados e preferências do usuário. 4. O sistema organiza os dados em gráficos, indicadores ou listas. 5. O sistema apresenta o <i>dashboard</i> ao usuário.
Fluxo Alternativo	Caso não existam dados suficientes, o sistema exibe uma mensagem informando que ainda não há informações disponíveis.
Pós-condições	O usuário visualiza informações analíticas sobre seu perfil de utilização.
Regras de Negócio	O <i>dashboard</i> deve atualizar os dados analíticos de forma automática com base nas interações recentes.

APÊNDICE B – Protótipo

Este apêndice apresenta as telas do protótipo atual do *Salsilauncher*. As figuras demonstram a organização visual planejada para a aplicação, incluindo autenticação de usuários, navegação principal, biblioteca de jogos e funcionalidades de cadastro.

A Figura 4 apresenta a tela de login da aplicação. Nessa tela, o usuário informa e-mail e senha para acessar sua conta no sistema.

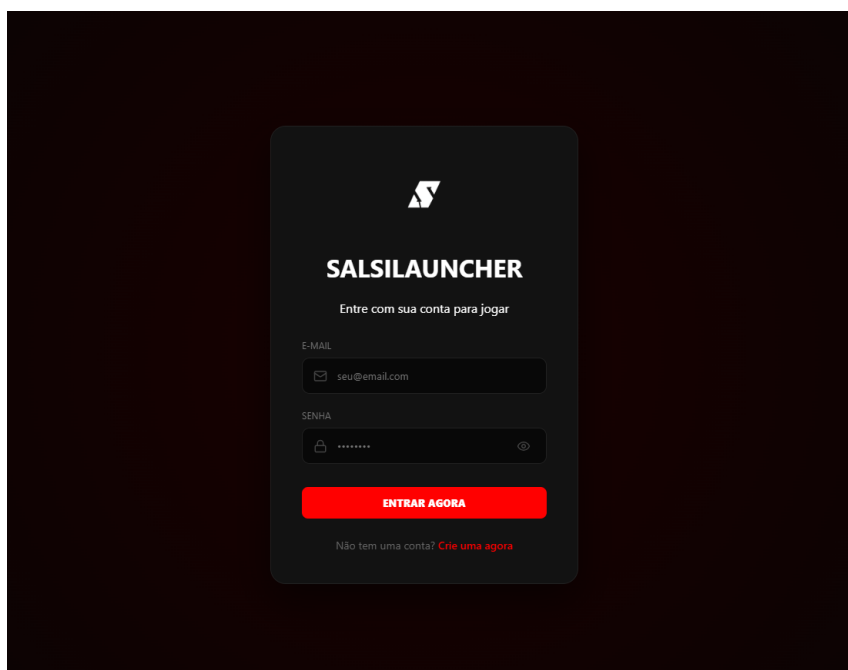


Figura 4 – Protótipo da tela de login

A Figura 5 apresenta a tela de criação de conta. Nela, o usuário informa nome, e-mail e senha para realizar seu cadastro na plataforma.

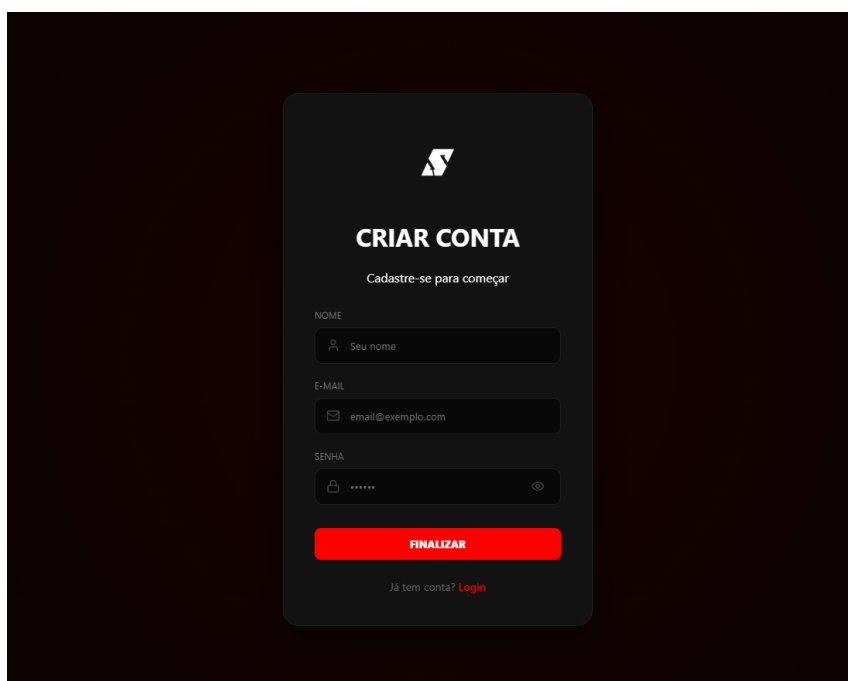


Figura 5 – Protótipo da tela de criação de conta

A Figura 6 apresenta a tela inicial da aplicação após o login. Essa tela exibe a área principal da biblioteca, a barra de pesquisa, o menu lateral e a identificação do usuário autenticado.

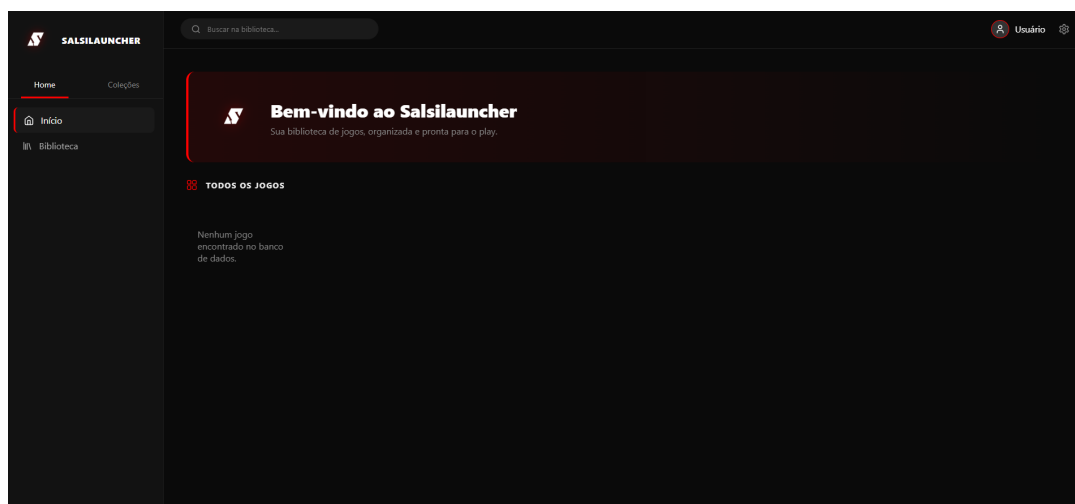


Figura 6 – Protótipo da tela inicial da biblioteca

A Figura 7 apresenta o menu lateral com a aba inicial selecionada. Esse componente concentra os principais caminhos de navegação relacionados à biblioteca do usuário.

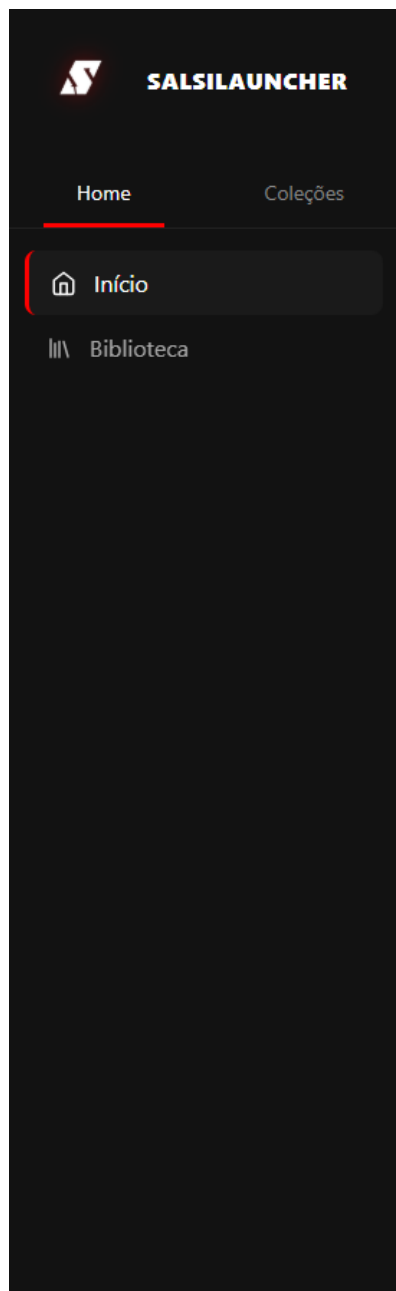


Figura 7 – Protótipo do menu lateral na aba inicial

A Figura 8 apresenta o menu lateral com a aba de coleções selecionada. Essa área permite acessar recursos como favoritos, criação de novo jogo e escaneamento de pastas.

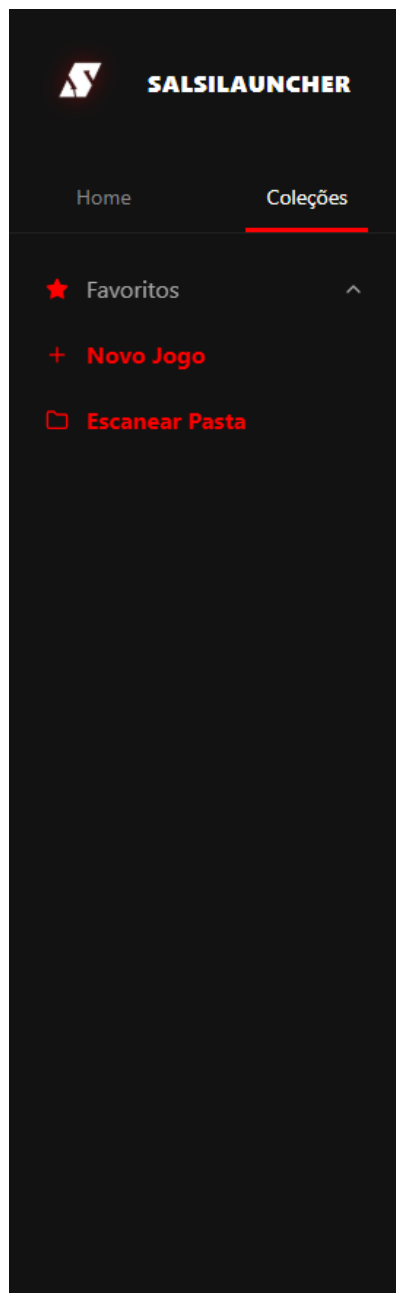


Figura 8 – Protótipo do menu lateral na aba de coleções

A Figura 9 apresenta a tela de cadastro manual de jogos. Nessa tela, o usuário pode informar dados como título, gênero, URL da capa e caminho do executável, além de visualizar uma prévia do jogo antes de salvá-lo.

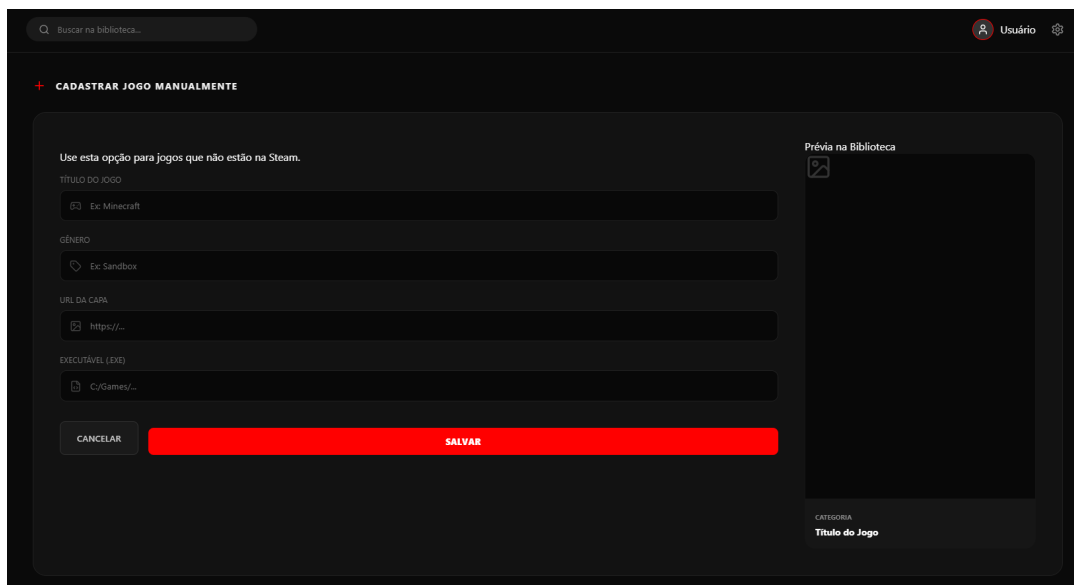


Figura 9 – Protótipo da tela de cadastro manual de jogo

A Figura 10 apresenta a tela de escaneamento de pasta. Essa funcionalidade permite que o usuário informe um diretório do computador para que o sistema busque jogos ou executáveis presentes naquela pasta.

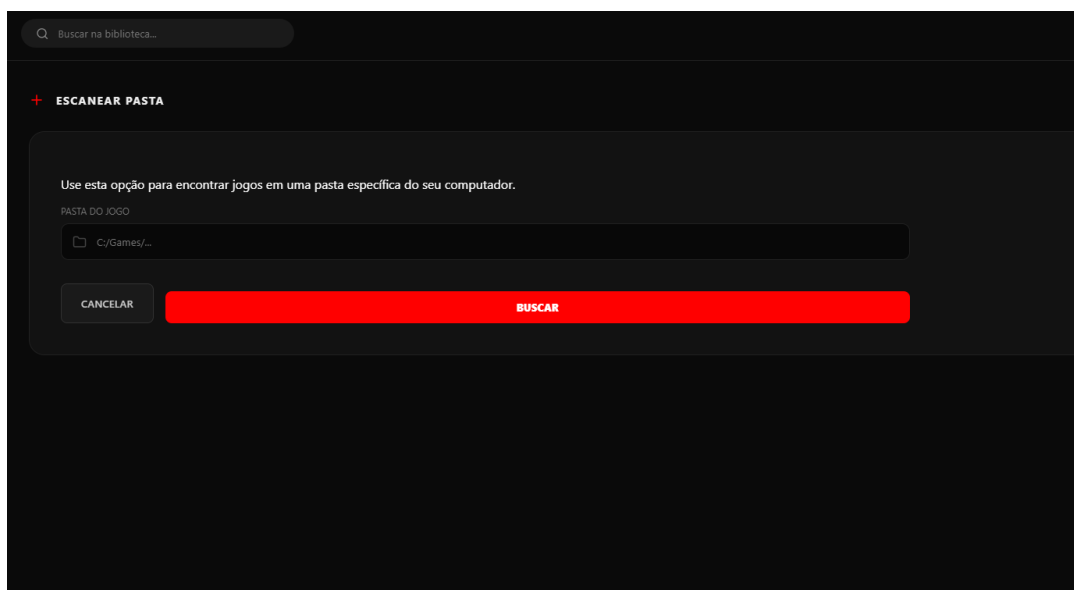


Figura 10 – Protótipo da tela de escaneamento de pasta

APÊNDICE C – Tabelas de Casos de Teste

Este apêndice apresenta os casos de teste planejados para validar as funcionalidades do *Salsilauncher*. Os testes foram organizados conforme os módulos do sistema, contemplando biblioteca, execução

e monitoramento, integrações externas, sistema de recomendação recomendação, funcionalidades sociais, *dashboard* analítico, coleções e requisitos funcionais e não funcionais.

3.1. Gerenciamento da Biblioteca

A [Tabela 27](#) detalha o procedimento de teste voltado à validação do cadastro manual de jogos na biblioteca do *Salsilauncher*.

Tabela 27 – Caso de teste CT-01

Identificador	CT-01
Funcionalidade	Cadastro de jogo
Objetivo	Verificar se um jogo pode ser adicionado à biblioteca.
Pré-condição	Nenhuma.
Entrada	Nome do jogo e caminho válido do executável.
Resultado esperado	O jogo deve ser salvo e exibido na biblioteca do usuário.
Exceção (Caminho Triste)	Se o caminho do executável for inválido ou inexistente, o sistema deve barrar o cadastro e exibir uma mensagem de erro ("Caminho do arquivo não encontrado").

A [Tabela 28](#) apresenta a especificação do teste elaborado para verificar a atualização cadastral das informações de um jogo já existente na biblioteca.

Tabela 28 – Caso de teste CT-02

Identificador	CT-02
Funcionalidade	Edição de jogo
Objetivo	Verificar a alteração dos dados cadastrados de um jogo.
Pré-condição	Jogo já cadastrado e visível na biblioteca.
Entrada	Alteração de informações previamente cadastradas (ex: nome ou imagem da capa).
Resultado esperado	Os dados devem ser atualizados corretamente no banco e na interface.
Exceção (Caminho Triste)	Se o formulário for submetido com campos obrigatórios em branco, a edição não deve ser concluída.

A [Tabela 29](#) descreve o caso de teste direcionado à validação da remoção de um jogo da biblioteca do usuário.

Tabela 29 – Caso de teste CT-03

Identificador	CT-03
Funcionalidade	Remoção de jogo
Objetivo	Verificar a exclusão de um jogo da biblioteca.
Pré-condição	Jogo cadastrado no sistema.
Entrada	Seleção de um jogo existente e confirmação de exclusão.
Resultado esperado	O jogo deve ser removido do banco de dados e deixar de aparecer na biblioteca.
Exceção (Caminho Triste)	N/A

A [Tabela 30](#) especifica o teste elaborado para verificar a eficiência dos mecanismos de busca e filtragem de jogos cadastrados na biblioteca.

Tabela 30 – Caso de teste CT-04

Identificador	CT-04
Funcionalidade	Pesquisa de jogos
Objetivo	Verificar a busca por jogos cadastrados.
Pré-condição	Biblioteca contendo um ou mais jogos cadastrados.
Entrada	Nome completo ou parcial de um jogo inserido na barra de pesquisa.
Resultado esperado	O sistema deve filtrar e exibir apenas os jogos compatíveis com a pesquisa.
Exceção (Caminho Triste)	Se o termo pesquisado não corresponder a nenhum jogo, exibir mensagem "Nenhum jogo encontrado".

3.2. Execução e Monitoramento

A [Tabela 31](#) apresenta o caso de teste estruturado para validar a correta inicialização do arquivo executável associado a um jogo cadastrado.

Tabela 31 – Caso de teste CT-05

Identificador	CT-05
Funcionalidade	Execução de jogo
Objetivo	Verificar a inicialização do executável associado ao jogo.
Pré-condição	Caminho do executável íntegro e arquivo existente no disco.
Entrada	Comando de execução (clique em "Jogar").
Resultado esperado	O processo do jogo deve ser iniciado corretamente no sistema operacional.
Exceção (Caminho Triste)	Se o executável tiver sido movido, exibir erro alertando que o jogo não pode ser executado.

A [Tabela 32](#) detalha a verificação da contagem e persistência automática do tempo de execução das sessões de jogo pelo *Salsilauncher*.

Tabela 32 – Caso de teste CT-06

Identificador	CT-06
Funcionalidade	Registro de tempo de uso
Objetivo	Verificar o monitoramento da duração da sessão de jogo.
Pré-condição	Execução do jogo através do CT-05 realizada com sucesso.
Entrada	Acompanhamento do processo e encerramento do jogo.
Resultado esperado	O tempo total em minutos ou horas deve ser somado e salvo no perfil do usuário.
Exceção (Caminho Triste)	Se o <i>launcher</i> for fechado de forma forçada antes do jogo, o tempo parcial registrado até a falha deve ser persistido, se possível.

A [Tabela 33](#) especifica a validação do mecanismo de prevenção de redundância, desenvolvido para evitar a abertura de múltiplas instâncias de um mesmo jogo.

Tabela 33 – Caso de teste CT-07

Identificador	CT-07
Funcionalidade	Prevenção de dupla execução
Objetivo	Verificar se o sistema impede a inicialização de uma nova instância de um jogo já aberto.
Pré-condição	O jogo já foi iniciado e seu processo está ativo no sistema operacional.
Entrada	Tentativa de executar novamente o mesmo jogo.
Resultado esperado	O sistema deve bloquear a ação, não iniciar um novo processo e exibir uma notificação/alerta informando: "Este jogo já está em execução".
Exceção (Caminho Triste)	Se o <i>launcher</i> perder o rastreamento do processo e permitir a execução dupla, o teste deve falhar.

3.3. Integração com APIs Externas

A [Tabela 34](#) apresenta o procedimento de teste para verificação do consumo de serviços externos e preenchimento automático de metadados dos jogos.

Tabela 34 – Caso de teste CT-08

Identificador	CT-08
Funcionalidade	Importação de metadados
Objetivo	Verificar a obtenção automática de informações do jogo.
Pré-condição	Conexão com a internet ativa e API de metadados operante.
Entrada	Captura dos metadados de um jogo com título presente na base de dados externa.
Resultado esperado	Nome, descrição, capa e categorias devem ser preenchidos automaticamente.
Exceção (Caminho Triste)	Se a API estiver indisponível ou retornar <i>timeout</i> , permitir inserção manual dos metadados sem travar o <i>launcher</i> .

A [Tabela 35](#) descreve o caso de teste voltado à validação da sincronização de bibliotecas a partir de contas vinculadas da plataforma *Steam*.

Tabela 35 – Caso de teste CT-09

Identificador	CT-09
Funcionalidade	Sincronização da biblioteca <i>Steam</i>
Objetivo	Verificar a importação de jogos de uma conta vinculada.
Pré-condição	Perfil <i>Steam</i> configurado como público.
Entrada	SteamID válido informado pelo usuário.
Resultado esperado	Os jogos da biblioteca <i>Steam</i> informada devem ser importados para o <i>Salsilauncher</i> .
Exceção (Caminho Triste)	Se o SteamID for inválido ou privado, exibir mensagem informando que não foi possível realizar a leitura dos jogos.

3.4. Sistema de Recomendação

A [Tabela 36](#) especifica o teste elaborado para avaliar a geração de recomendações personalizadas com base nas preferências e histórico do usuário.

Tabela 36 – Caso de teste CT-10

Identificador	CT-10
Funcionalidade	Geração de recomendações
Objetivo	Verificar a geração de sugestões com base no perfil do usuário.
Pré-condição	Usuário com histórico de horas jogadas e categorias definidas no banco de dados.
Entrada	Acesso à aba de recomendações.
Resultado esperado	O sistema deve apresentar uma lista de sugestões compatíveis com o perfil do usuário.
Exceção (Caminho Triste)	Se o usuário não possuir dados, exibir uma lista genérica de jogos populares.

3.5. Sistema Social

A [Tabela 37](#) apresenta o caso de teste direcionado ao estabelecimento e confirmação do vínculo de amizade entre usuários cadastrados.

Tabela 37 – Caso de teste CT-11

Identificador	CT-11
Funcionalidade	Adição de amigos
Objetivo	Verificar a criação de vínculos entre usuários.
Pré-condição	Ambos os usuários estão cadastrados na plataforma.
Entrada	Envio de solicitação e respectivo aceite.
Resultado esperado	Os usuários devem aparecer mutuamente em suas listas de amigos.
Exceção (Caminho Triste)	Se o usuário tentar adicionar um ID inexistente, exibir "Usuário não encontrado".

A [Tabela 38](#) detalha o procedimento de teste para verificação da exibição em tempo real das atividades públicas registradas pela rede de amigos.

Tabela 38 – Caso de teste CT-12

Identificador	CT-12
Funcionalidade	Visualização de atividades
Objetivo	Verificar a exibição das atividades em tempo real de amigos.
Pré-condição	Usuário possui amigos cadastrados com atividades recentes.
Entrada	Navegação para a aba social.
Resultado esperado	O sistema deve exibir informações públicas como "Jogando agora" ou "Último jogo acessado".
Exceção (Caminho Triste)	Se o amigo alterar a privacidade para "Invisível", sua atividade não deve ser exibida.

3.6. Dashboard Analítico

A Tabela 39 especifica o teste elaborado para validar a renderização e o cálculo dos indicadores e gráficos do *dashboard* analítico.

Tabela 39 – Caso de teste CT-13

Identificador	CT-13
Funcionalidade	<i>Dashboard</i> analítico
Objetivo	Verificar a geração das estatísticas e métricas do usuário.
Pré-condição	Possuir registros válidos de tempo de uso associados à conta.
Entrada	Acesso ao menu de estatísticas/ <i>dashboard</i> .
Resultado esperado	O sistema deve apresentar gráficos e indicadores corretamente renderizados.
Exceção (Caminho Triste)	Caso o cálculo estatístico seja custoso, a tela deve exibir um <i>loader</i> até a conclusão dos dados.

3.7. Gerenciamento de Pastas e Coleções

A Tabela 40 apresenta o caso de teste para verificação do rastreamento e mapeamento automático de jogos em diretórios locais do computador.

Tabela 40 – Caso de teste CT-14

Identificador	CT-14
Funcionalidade	Escanear pasta
Objetivo	Verificar o mapeamento automático e adição em massa de jogos a partir de um diretório.
Pré-condição	Pasta contendo um ou mais diretórios ou executáveis de jogos.
Entrada	Caminho da pasta selecionado pelo usuário.
Resultado esperado	O sistema deve identificar jogos, listá-los para confirmação e adicioná-los à biblioteca.
Exceção (Caminho Triste)	Se a pasta estiver vazia ou sem permissão de leitura, exibir mensagem de erro apropriada.

A Tabela 41 detalha a validação da funcionalidade de criação e persistência de coleções personalizadas para organização da biblioteca.

Tabela 41 – Caso de teste CT-15

Identificador	CT-15
Funcionalidade	Criar coleção
Objetivo	Verificar a capacidade de criar grupos personalizados para categorizar jogos.
Pré-condição	Nenhuma.
Entrada	Inserção de um nome válido para a nova coleção.
Resultado esperado	A coleção deve ser criada, persistida e exibida na interface.
Exceção (Caminho Triste)	Não deve ser permitido salvar uma coleção com nome em branco.

A [Tabela 42](#) especifica o teste elaborado para verificar a alteração de nomes e atualização de títulos vinculados a uma coleção existente.

Tabela 42 – Caso de teste CT-16

Identificador	CT-16
Funcionalidade	Editar coleção
Objetivo	Verificar a alteração das propriedades e do conteúdo de uma coleção.
Pré-condição	Uma coleção já existente.
Entrada	Novo nome e/ou alteração dos jogos associados.
Resultado esperado	O nome e os jogos vinculados devem refletir as alterações realizadas.
Exceção (Caminho Triste)	Não deve ser permitido salvar uma coleção com nome em branco.

A [Tabela 43](#) descreve o caso de teste focado na exclusão de agrupamentos de coleções sem prejuízo aos jogos contidos na biblioteca principal.

Tabela 43 – Caso de teste CT-17

Identificador	CT-17
Funcionalidade	Remover coleção
Objetivo	Verificar a exclusão de uma coleção sem remover os jogos associados.
Pré-condição	Coleção existente selecionada na interface.
Entrada	Clique no botão de exclusão da coleção.
Resultado esperado	A coleção deve desaparecer da interface e os jogos devem permanecer na biblioteca.
Exceção (Caminho Triste)	N/A

A [Tabela 44](#) apresenta a validação do mecanismo de busca e filtragem aplicado especificamente às coleções do usuário.

Tabela 44 – Caso de teste CT-18

Identificador	CT-18
Funcionalidade	Pesquisa de coleções
Objetivo	Verificar a funcionalidade de busca pelo nome das coleções.
Pré-condição	Múltiplas coleções cadastradas.
Entrada	Digitação completa ou parcial do nome da coleção.
Resultado esperado	A interface deve exibir apenas as coleções compatíveis com a pesquisa.
Exceção (Caminho Triste)	Se não houver correspondência, exibir "Nenhuma coleção encontrada".

3.8. Validação de Requisitos Não Funcionais

A [Tabela 45](#) especifica o procedimento de teste não funcional focado em avaliar o tempo de resposta e desempenho de consultas no *backend*.

Tabela 45 – Caso de teste CT-19

Identificador	CT-19
Funcionalidade	Performance de Consulta no <i>Backend</i>
Objetivo	Garantir que <i>endpoints</i> pesados respondam dentro de limites aceitáveis.
Pré-condição	Banco de dados populado com uma carga média realista de dados.
Entrada	Requisição GET para geração de estatísticas.
Resultado esperado	A API deve processar e devolver a resposta em tempo aceitável (inferior a 1,5 segundos).
Exceção (Caminho Triste)	Caso a resposta ultrapasse o tempo estipulado, o teste automatizado deve falhar, indicando necessidade de otimização.